

RAHGRec: A Role-Aware Hypergraph-Based Framework for Project–Library Recommendation in Python Ecosystem with Knowledge-Graph Relations

ABHINAV JAMWAL, Indian Institute of Technology Roorkee, India

SANDEEP KUMAR, Indian Institute of Technology Roorkee, India

The Project–Library relationship plays a central role in modern-day software development but choosing libraries which are not just compatible but also well-suited to the particular setting of the project has always been an issue. Most of today’s project–library recommendations systems rely on the co-usage information extracted from history or the latent representations derived through pairwise interactions. These systems have proven to perform excellently in environments where there is abundant data but they tend to face challenges distinguishing between common co-adoption and actual compatibility, thus enhancing false correlations.

In this paper, we propose RAHGRec, a role-aware hypergraph-based recommendation framework that integrates higher-order co-use patterns with semantic relationships from a knowledge graph to support Project library recommendation. RAHGRec models project–library usage as hypergraphs to reflect how groups of libraries are adopted together, and it directly leverages typed semantic relations such as dependency links, topical similarity, and maintainer-related signals through role-aware propagation mechanisms. By restricting higher-order message passing according to these semantic roles, the framework can differentiate between co-usage that is merely frequent in the data and adoption that truly fits the surrounding context. We evaluated RAHGRec on a large-scale Python ecosystem benchmark comprising 12,421 projects, 963 libraries, and over 121,000 project–library interactions enriched with contextual metadata. The results show that RAHGRec consistently surpasses the state-of-the-art baselines across all reported metrics, including precision, recall, F1, MRR, and coverage. The advantage is most pronounced under the sparse and long-tail interaction settings, which underscores the value of role-aware, semantically constrained higher-order modeling for producing reliable and context-sensitive library recommendations when observed project–library histories are limited.

CCS Concepts: • **Software and its engineering** → **Software organization and properties**; • **Computing methodologies** → **Machine learning algorithms**.

Additional Key Words and Phrases: Software library ecosystem, Project library recommendation, Role-aware hypergraph modeling, Knowledge graph-driven relations, Machine learning.

1 Introduction

The rapid growth of modern software development, particularly within open-source Python ecosystems, has intensified developers’ reliance on project libraries to accelerate development and enhance productivity. Python’s broad adoption is closely related to its rich ecosystem, which by 2026 includes more than 640,000 libraries and nearly 7 million published releases on the Python Package Index (PyPI)¹. Despite the abundance of reusable code

¹<https://pypi.org/>

Authors’ Contact Information: Abhinav Jamwal, abhinav_j@cs.iitr.ac.in, Indian Institute of Technology Roorkee, Roorkee, Uttarakhand, India; Sandeep Kumar, sandeep.garg@cs.iitr.ac.in, Indian Institute of Technology Roorkee, Roorkee, Uttarakhand, India.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, or post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1557-7392/2026/9-ART

<https://doi.org/10.1145/3846193>

available, the challenge of choosing libraries that not only fulfill the project’s functionalities but also integrate well with the expected levels of dependencies, domain, and maintenance becomes even more difficult.

This challenge has become sharper with the rise of tools built around large language models (LLMs), retrieval-augmented generation, and autonomous coding agents. Modern development assistants can suggest code snippets and the libraries they rely on from a partial project context or even a natural-language description. But in many cases, those suggestions are driven by surface-level signals like keyword similarity or popularity. They do not reliably reflect how dependencies interact, why one library is a better fit than another in a given setting, or what ecosystem-specific best practices imply. Therefore, developers still need to examine the recommendation list, filter out unsuitable options, and decide whether the suggested libraries are actually relevant to their application. In this regard, this paper addresses the above problem by arguing that library recommendation signals should rely on explicit semantic relations, rather than only on implicit similarities.

Moreover, libraries often appear in groups or clusters. There are specific combinations of libraries that occur repeatedly since they provide additional functionality, use the same architectural style, or conform to domain conventions. Such “stacks” of libraries are not just simple one-to-one correspondences but imply higher-level structures like common building blocks and well-known design patterns. To capture such structures, it is necessary to use a modeling technique that allows for the interaction of multiple libraries simultaneously, taking into account the context.

Most existing library algorithms in the library domain mainly consider the interactions between projects and libraries as basic pairs. Collaborative filtering(CF) [2, 5, 12] and matrix factorization(MF) [11, 19] perform very well under dense datasets; yet, their performance will deteriorate as datasets become sparse, and they fail to model the more complex structures arising from the frequent use of multiple libraries in coordination [31]. On the contrary, hypergraph-based recommendation methods directly model the structures in high order [3], and more recent multiscale architectures like MsRec [13] enhance the learning of high-level representations by simultaneously modeling the interactions at different scales. Furthermore, KG-GNNs can provide the recommendation models with extra context such as dependency relationships, category labels, and semantic relatedness [21].

Even with these advances, there appears to be a remaining methodological gap. Hypergraph-based models usually focus on structural patterns of co-usage, but they mostly treat interactions as if they were all semantically the same, without clearly accounting for how contextual relationships shape higher-order aggregation. In contrast, KG-GNN-based approaches place stronger emphasis on semantic information, but they usually rely on pairwise graph structures. This limits their ability to capture coherent groups of libraries and more complex adoption patterns. As a result, existing methods often capture only one side of the problem: either higher-order structure without enough semantic guidance, or semantic context without a sufficient model of higher-order co-usage.

In practice, higher-order co-usage signals are informative only insofar as the underlying relationships generalize across project contexts. Libraries may frequently co-occur due to templates, transient trends, or incidental dependencies, yet such patterns may not transfer reliably across project contexts when dependency structures, domains, or maintenance assumptions differ. This observation motivates a semantically constrained formulation of higher-order recommendation, in which contextual relations explicitly guide how higher-order evidence is constructed and propagated, rather than treating all co-usage signals as equally informative.

These contextual relations do not contribute to recommendation in the same way. A dependency link captures compatibility constraints among libraries, a topic link reflects functional or domain relatedness, and a maintainer link provides maintenance-level context. We therefore treat each typed knowledge-graph relation as a distinct semantic role that defines how evidence should be grouped, propagated, and weighted. Under this view, role-aware modeling means preserving these relation-specific roles during hypergraph construction and propagation, rather than merging all semantic links into a single homogeneous signal.

The main motivation for this research is illustrated in Figure 1. MsRec [13] employs multiscale hypergraphs to represent higher-order co-usage, but it considers all interactions as semantically equivalent and does not

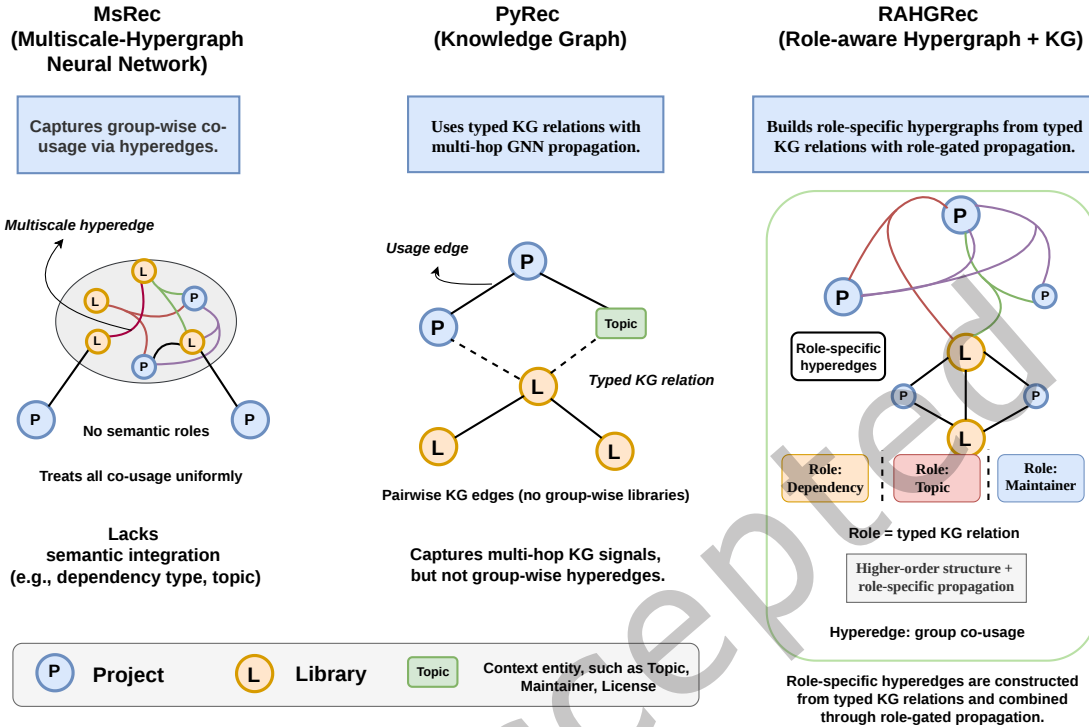


Fig. 1. Conceptual comparison of project-library recommendation paradigms. MsRec [13] captures higher-order co-usage with multiscale hypergraphs but does not distinguish semantic roles. PyRec [21] uses typed knowledge-graph relations with GNN-based multi-hop propagation, but models them through pairwise KG edges rather than group-wise hyperedges. RAHGRec constructs role-specific hyperedges from typed KG relations, where roles denote semantic relation types such as dependency, topic, and maintainer, and applies role-gated propagation for higher-order semantic aggregation.

control information diffusion along typed relationships. PyRec [21] introduces contextual information through usage edges, topic nodes, and dependency edges, and its GNN-based propagation can capture multi-hop semantic signals in the knowledge graph. However, these signals are still propagated over pairwise KG paths and are not organized as explicit group-wise hyperedges. In contrast, RAHGRec treats typed semantic relations as role-specific hypergraph structures, allowing higher-order co-usage evidence to be constructed, propagated, and weighted under distinct semantic roles. This design preserves the strength of hypergraph-based group modeling while adding semantic control over how higher-order evidence contributes to recommendation.

Building on this design rationale, we present RAHGRec, a role-aware hypergraph learning framework for semantically constrained project-library recommendation, guided by a software knowledge graph. Our framework represents project–library usage with hypergraphs so that higher-order co-usage patterns are retained, and it augments this structure with typed semantic relations from a software knowledge graph, such as dependency links, topical relatedness, and maintainer-level signals. We use these relations both to construct role-specific hyperedges and to guide message propagation, allowing the model to weight co-usage evidence differently depending on

the underlying semantic context. This formulation works especially well in scenarios where frequency-based learning fails, and learning is vulnerable to being made stable via semantically guided representation learning.

To validate the proposed approach, we evaluate RAHGRec through experiments that examine recommendation effectiveness, propagation depth, semantic regulation, embedding sensitivity, and scalability under sparse and long-tail project-library interactions, where only part of the observed interaction history is available. The key contributions of this work are as follows:

- We propose RAHGRec, a relation-aware hypergraph neural framework for project-library recommendation that integrates higher-order co-usage modeling with typed semantic relation modeling, enabling joint reasoning over usage structure and contextual semantics.
- We introduce a role-aware hypergraph construction scheme that transforms relation-typed knowledge graph triples into a family of role-isolated hypergraphs, preserving semantic roles as first-class structures rather than collapsing heterogeneous relations into a single graph.
- We introduce a dual-channel propagation scheme that combines (i) usage-driven behavioral signals propagated over project-library adoption edges and (ii) higher-order semantic context propagated through role-specific hypergraphs. A learnable role-gating module further regulates how information from different roles is aggregated during message passing.
- We develop a multi-layer representation aggregation strategy based on layer-wise concatenation of embeddings across propagation depths, enabling multi-scale representations that remain stable under sparse and long-tail adoption settings.
- We conducted an extensive empirical evaluation on a large-scale Python library dataset, demonstrating consistent improvements over state-of-the-art baseline methods through statistical significance testing and ablation analysis.
- We provide a publicly available replication package (code, scripts, and processed data) through Zenodo² and maintain an implementation repository on GitHub³ to support reproducibility.

The remainder of this paper is organized as follows. Section 2 discusses the related literature. Section 3 presents the motivating example. Section 4 details the proposed RAHGRec approach. Section 5 presents the experimental setup and evaluation results. Section 6 discusses key findings and implications of the design. Finally, Section 7 concludes the article and outlines potential directions for future work.

2 Related Work

Recommendation techniques have been extensively studied in software engineering to support tasks such as defect identification [41], developer recommendation [17, 37], API and code snippet recommendation [28, 40], project-library recommendation [4, 11, 20, 27, 32, 38], and permission recommendation [24]. The library recommendation task is especially interesting amongst these, as the modern software development process makes extensive use of external libraries for quick development and efficient reuse of existing capabilities. As a result, choosing suitable libraries directly affects the quality, maintainability, and long-term evolution of the surrounding ecosystem.

Collaborative Filtering and Matrix Factorization. Earlier library recommendation approaches have mostly relied on collaborative filtering (CF) and matrix factorization (MF). For example, LibRec [32] combines association rule mining with CF to make recommendations for Java project libraries, while CrossRec [27] leverages CF by making use of cross-project patterns. Although these models yield accurate recommendations in cases where there is a lot of interaction data, they become less efficient in cases where data is sparse [26]. As shown in

²<https://zenodo.org/records/18450502>

³<https://github.com/ABhiITRoorkee/RAHGRec>

Table 1. Comparisons of existing recommendation approaches in project-library with the RAHGRec framework.

Modeling Capability	LibRec [32]	CrossRec [27]	LibSeek [11]	GRec [20]	HCF [3]	PyRec [21]	MsRec [13]	RAHGRec
Pairwise project-library modeling	✓	✓	✓	✓	✓	✓	✓	✓
Multi-hop graph/KG propagation	✗	✗	✗	✓	✗	✓	✗	✓
Explicit higher-order co-adoption modeling via hypergraphs	✗	✗	✗	✗	✓	✗	✓	✓
Group-wise interaction preservation	✗	✗	✗	✗	✓	✗	✓	✓
Knowledge graph semantic integration	✗	✗	✗	✗	✗	✓	✗	✓
Supplementary contextual KG entities	✗	✗	✗	✗	✗	✓	✗	✓
Typed KG relation modeling	✗	✗	✗	✗	✗	✓	✗	✓
Role-specific hypergraph construction	✗	✗	✗	✗	✗	✗	✗	✓
Role-isolated semantic representation	✗	✗	✗	✗	✗	✗	✗	✓
Semantic regulation of higher-order propagation	✗	✗	✗	✗	✗	✗	✗	✓
Role-gated aggregation mechanism	✗	✗	✗	✗	✗	✗	✗	✓
Joint explicit hypergraph and KG semantic reasoning	✗	✗	✗	✗	✗	✗	✗	✓

Table 1 (see features *Explicit higher-order co-adoption modeling via hypergraphs*, *Group-wise interaction preservation*, and *Knowledge graph semantic integration*), CF- and MF-based approaches are mainly limited to pairwise project-library modeling. They do not explicitly preserve group-wise co-adoption patterns, nor do they integrate knowledge-graph-based semantic relations.

LibSeek [11] partially mitigates popularity bias by proposing a diversity-aware MF framework with adaptive weighting. Although this design enhances recommendation diversity, LibSeek remains a pairwise matrix-factorization method and does not explicitly model higher-order co-adoption, group-wise interactions, or typed KG relations, as shown in Table 1.

Graph-Based Recommendation Models. To better capture rich representations, researchers have proposed graph-based recommendation methods. For instance, GRec [20] represents project-library interactions as a bipartite graph and uses graph neural networks (GNNs) to learn both direct and indirect connections. Relative to CF and MF, graph-based methods strengthen relational representation through multi-hop graph propagation. However, as shown in Table 1, these models remain based on pairwise graph edges and do not explicitly incorporate knowledge-graph-derived semantic context such as dependency roles, project topics, or maintainer information, despite evidence that such signals are important in related recommendation tasks [39, 40].

Hypergraph-Based Recommendation Models. To overcome the representational limitations of pairwise graphs, hypergraph-based representation learning has emerged as a promising direction for modeling higher-order relations among multiple entities [6, 8, 22, 33]. HCF [3] introduced similarity-driven hypergraphs to model higher-order relationships among multiple entities, alleviating information loss and oversmoothing issues common in standard GNNs. Building on this line of work, MsRec [13] proposed a multiscale hypergraph framework that captures library co-use patterns across different granularities. As shown in Table 1, these methods preserve higher-order co-adoption and group-wise interactions through hypergraphs. However, they treat co-usage relations as structurally homogeneous and do not explicitly incorporate typed KG relations, role-specific hypergraph construction, or semantic regulation of higher-order propagation. As a result, semantically incompatible co-usage patterns may exert equal influence during aggregation.

Knowledge Graph-Enhanced Recommendation. In parallel, knowledge graph (KG)-based techniques have demonstrated strong effectiveness in representing semantic relationships in a range of software engineering tasks [10, 23, 29, 34]. Motivated by these advances, PyRec [21] incorporates KG-derived contextual information, such as dependencies, topics, and authors, into bipartite project-library graphs to improve recommendation quality. As shown in Table 1, PyRec integrates supplementary KG entities, typed KG relations, and multi-hop

GNN-based propagation. However, its propagation is still performed over pairwise KG edges. Thus, PyRec captures multi-hop semantic relationships through pairwise KG paths, but it does not explicitly construct group-wise hyperedges or role-specific hypergraph channels for higher-order semantic aggregation.

Together, the existing approaches address different parts of the recommendation problem. CF- and MF-based methods model low-order project–library interactions but struggle with sparse data and group-wise adoption patterns. Graph-based methods improve relational propagation but remain based on pairwise edges. Hypergraph-based methods preserve higher-order co-adoption but generally treat co-usage relations as semantically homogeneous. KG-enhanced methods, such as PyRec, incorporate contextual semantics through typed KG relations and multi-hop propagation, but they do not explicitly organize such semantics into role-specific hypergraph structures. These limitations, summarized in Table 1, motivate a unified framework that can jointly model group-wise co-adoption and regulate higher-order propagation through typed semantic roles.

To address this gap, we introduce RAHGREC, a role-aware hypergraph recommendation framework that jointly models higher-order co-usage patterns and knowledge-graph-derived semantic relations within a unified neural architecture. By embedding typed semantic signals directly into hypergraph construction and propagation, RAHGREC enables role-aware, context-sensitive message passing, yielding robust and semantically compatible library recommendations, particularly in sparse and large-scale settings.

3 Motivating Example

To motivate the necessity of semantically constrained higher-order library recommendations, consider an oversimplified representation of the software ecosystem shown in Figure 2. It involves some Python software projects (P1–P7) and some candidate libraries (L1–L4), along with their interaction facts and semantic information. Such a model is representative of modern software development when developers employ artificial intelligence-enabled systems and self-driving coding agents that provide recommendations for code snippets and dependencies to use. These systems are able to provide suggestions quickly, but they do not have any knowledge about the entire ecosystem and its constraints, thus, developers are forced to validate the proposed solutions themselves.

In practice, choosing the project–library is about much more than just seeing what is out there. The developers must judge whether a library truly satisfies the functional needs of a project, whether it brings about transitive or risky dependencies that are hard-to-manage, and whether it fits with common patterns in the domain, expected maintenance practices, and the long-term direction of the broader ecosystem. These decisions get even harder in big, fast-changing ecosystems like Python, where we have little hard data on how pieces work together, libraries grow in different directions, and projects that look alike on the surface can actually rely on very different assumptions about their dependencies.

Figure 2 illustrates two complementary signals that are both crucial for dependable project–library recommendation, yet are seldom considered together in existing models. First, project–project links represent semantic similarity rather than dependency relations; the topic-set Jaccard label indicates that this similarity is computed from the overlap between project topic sets. Second, dependency relations are used only for library-level relations, such as dependencies between libraries. Thus, links such as $P2-P5$ should be interpreted as topic-based semantic similarity, not as project–project dependency.

- (1) **Higher-order co-adoption patterns.** In the real-world software projects, libraries are usually adopted as cohesive sets rather than as isolated components. For example, projects P2 and P3 might consistently rely on the bundle {L2, L3}, reflecting a recurring architectural pattern or a domain-focused technology stack. Such group-level adoption patterns provide richer contextual information than individual library pairs, exposing conventions, templates, and implicit best practices that remain obscured when project–library relationships are considered one at a time.

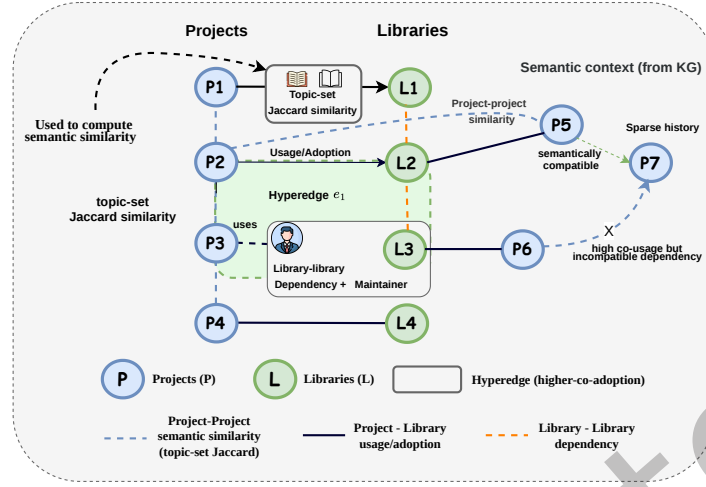


Fig. 2. Motivating example illustrating the higher-order project–library co-adoption and typed semantic relations used by the proposed approach. The hyperedges preserve group-wise co-adoption, while role-aware gating regulates propagation to prevent the semantically incompatible signals from influencing recommendations. Project–project links denote topic-set Jaccard similarity, not dependency relations, and dependency relations are shown only at the library level.

- (2) **Typed semantic relations.** Co-adoption by itself does not provide enough evidence to judge whether a library is suitable for a new or rarely observed project (e.g., P7). Typed semantical relations can help in giving more context on how well the libraries can be compatible to each other in terms of dependency relationship, topic compatibility between the libraries and projects, and others like having the same maintainers or meta information. Such semantical relations are important since sometimes libraries can occur in one package at the same time due to short-term popularity, dependent relationships, or other past reasons; however, they might not fit each other at all.

Most existing library recommendation methods tend to consider only one of these signals at a time. Pairwise recommenders built on collaborative filtering or matrix factorization fail to capture adoption patterns at the bundle level and often perform poorly when interactions are sparse. Hypergraph-based methods explicitly model higher-order co-adoption, but usually treat all co-usage relations as structurally uniform, allowing semantically incompatible libraries to exert equal influence during propagation. Conversely, knowledge-graph-based recommenders capture rich semantic relationships, yet their dependence on pairwise graph structures restricts how well they can model coherent library bundles and the higher-order contexts in which libraries are actually adopted. As a result, recommendations produced by structure-only or semantic-only methods may appear plausible while lacking contextual compatibility.

However, the proposed approach is able to overcome such limitations through the integration of higher-order co-adoption patterns and typed semantic relations in one unified framework of role-aware hypergraphs. The co-adoption of libraries in one project leads to the formation of hyperedges that capture the context of group interactions, whereas the semantic relations, obtained from the software knowledge graph, are used as role-based propagation paths that control the transfer of information, as depicted in Figure 2. For example, in cases where the target project, such as P7, has very little interaction history but is semantically similar to projects P2 and P5, the RAHGRec approach will take into consideration the higher-order co-adoption signals and weigh them according

to their semantic similarity, such as having the same topics or dependencies. However, co-usage patterns that are frequent but not semantically consistent, for example, due to a different dependency structure, have less weight assigned by the role-aware gating function.

By linking complex adoption patterns to clear semantic constraints, RAHGRec enables library recommendations to remain data-driven while also being grounded in context and straightforward to explain. This capability is particularly important in modern AI-assisted development workflows, where candidate dependencies can be generated automatically, but principled semantics-aware reasoning is required to ensure compatibility, maintainability, and robustness, especially under sparse and long-tail conditions.

4 RAHGRec Approach

RAHGRec(Role-Aware HyperGraph Recommendation) is a role-aware hypergraph learning framework for project-library recommendation that integrates (i) observed project-library adoption behavior and (ii) structured semantic signals encoded in a software knowledge graph. The key design principle is to *preserve semantic roles* (e.g., dependency-oriented, descriptive, authorship-related semantics) as first-class structures during representation learning, rather than collapsing all relations into a single homogeneous graph. Specifically, RAHGRec couples a usage-driven adjacency channel with a parallel family of role-specific hypergraph channels, allowing the model to propagate behavioral evidence and semantic context in a controlled and interpretable manner.

To facilitate reproducibility and independent validation, we make the implementation of **RAHGRec**, together with the processed dataset used in our experiments, publicly available in a consolidated replication package². We first describe how the software knowledge graph is constructed, after which Table 2 summarizes the core notation, Algorithm 1 outlines the computational workflow, and Figure 3 provides a visual overview of the complete RAHGRec pipeline. The remainder of this section presents the methodology in five sequential phases: unified input modeling and semantic signal preparation, role-aware hypergraph construction, dual-channel propagation, multi-layer representation aggregation, and relation-aware ranking and optimization.

4.1 Method Overview

Consider an illustrative software project, VisionPlus, designed for real-time image augmentation. This example is used only to explain the proposed method and does not refer to a specific external software project. The development team may shortlist libraries such as OpenCV for image processing, Albumentations for data augmentation, and NumPy for numerical computation. However, a practical recommendation must go beyond the frequency of co-usage: it must account for *compatibility constraints* (e.g. transitive dependencies) as well as *semantic suitability* (e.g. whether a library is aligned with the functional intent of the project). The proposed approach addresses this requirement by first constructing a typed software knowledge graph from project-library metadata and then jointly modeling adoption behavior and knowledge-graph semantics through role-aware hypergraph propagation.

The mathematical formulation that follows combines standard components from graph learning and recommender-system literature with the proposed role-aware hypergraph design. Standard components include embedding initialization, hypergraph incidence normalization, message passing, layer-wise representation aggregation, inner-product scoring, pairwise ranking loss, dropout, and ℓ_2 regularization. We cite the relevant prior work where these components are introduced in the corresponding phases. The original contribution of RAHGRec lies in transforming typed software knowledge-graph relations into role-specific hypergraphs, combining role-aware semantic propagation with project-library usage propagation, and jointly optimizing these signals for project-library recommendation.

4.1.1 Knowledge Graph Construction. The software knowledge graph used by RAHGRec is constructed from the processed project-library dataset and its associated project and library metadata, rather than assumed as a manually curated external resource. We create nodes for projects, libraries, and auxiliary metadata entities,

Table 2. Summary of Notations in RAHGRec

Notation	Description
$\mathcal{P}$	Set of software projects
$\mathcal{L}$	Set of project libraries
$\mathcal{E}_{\text{KG}}$	Set of auxiliary knowledge graph entities (e.g., authors, metadata)
$\mathcal{V}$	Unified entity set $\mathcal{P} \cup \mathcal{L} \cup \mathcal{E}_{\text{KG}}$
$\mathcal{D}$	Observed project–library interactions (implicit feedback)
$r_{\text{use}}, r_{\text{rev}}$	Usage relation and its inverse
$\mathcal{G}_{\text{KG}}$	Knowledge graph triples (h, r, t) including inverse relations
$\mathcal{R}$	Set of semantic relation roles derived from KG relations
$e_h^{(r)}$	Hyperedge anchored at head entity h under role r
$\mathcal{E}^{(r)}$	Set of hyperedges induced by role r
$\mathbf{H}^{(r)}$	Incidence matrix for role-specific hypergraph $\mathcal{H}^{(r)}$
$\tilde{\mathbf{H}}^{(r)}$	Degree-normalized incidence operator for role r
$\mathbf{A}$	Normalized adjacency matrix from usage relations only
$\mathcal{N}_{\text{use}}(v)$	Usage-based neighbors of entity v
$\mathbf{h}_v^{(0)}$	Initial trainable embedding of entity v
$\mathbf{q}_r$	Relation embedding for KG relation r
$\mathbf{M}_r$	Relation-specific projection matrix used in the TransR-style KG embedding component
$\mathbf{h}_v^{(k)}$	Embedding of entity v after k propagation layers
$\mathbf{H}^{(k)}$	Matrix stacking embeddings $\mathbf{h}_v^{(k)}$ for all $v \in \mathcal{V}$
β_r	Trainable logit controlling importance of role r
α_r	Softmax-normalized weight for semantic role r
$\mathbf{m}_{v,\text{use}}^{(k)}$	Usage-adjacency message for entity v at layer k
$\mathbf{m}_{v,\text{hg}}^{(k)}$	Role-aware hypergraph message for entity v at layer k
$\mathbf{m}_v^{(k)}$	Fused message for entity v at layer k
$\mathbf{W}_1^{(k)}, \mathbf{W}_2^{(k)}$	Trainable transformation matrices at layer k
$\sigma(\cdot)$	Nonlinear activation function (LeakyReLU)
ρ	Dropout probability applied to layer-wise updated embeddings
$\mathbf{z}_v$	Concatenated multi-layer embedding of entity v
$\tilde{\mathbf{z}}_v$	ℓ_2 -normalized embedding of entity v used for scoring
$\hat{y}_{p\ell}$	Predicted relevance score between project p and library ℓ
$\mathcal{L}_{\text{rank}}$	Pairwise ranking loss (BPR)
$\mathcal{L}_{\text{KG}}$	KG relational learning loss for typed semantic triples
$\mathcal{L}_{\text{reg}}$	ℓ_2 regularization loss on unnormalized embeddings $\mathbf{z}_v$
λ	Regularization coefficient
L	Number of propagation layers
d	Embedding dimensionality
K	Number of recommended libraries per project

including maintainers or authors, licenses, topics, keywords, and descriptive tags. Observed project–library adoption records are kept as usage facts, while metadata-derived semantic information is converted into typed KG triples. Specifically, dependency metadata generates library–library *depends-on* relations, maintainer and license fields generate *has-maintainer* and *has-license* relations, and topic, keyword, or tag fields generate descriptive semantic relations.

Similarity-based relations are constructed only between comparable entity types. Project–project similarity is computed from topic-set overlap, while library–library similarity is computed from metadata-set overlap. A similarity edge is retained only when its Jaccard score satisfies the corresponding threshold, denoted by α_p for project–project similarity and α_l for library–library similarity. After the typed triples are generated, inverse triples are added during Phase 1 to support balanced information flow. This construction makes the framework reproducible because the KG can be regenerated from the released processed metadata and scripts. It also supports application to other software ecosystems, provided that analogous package metadata, dependency manifests, or registry-level descriptions are available, when some metadata fields are absent, the method can still operate with the available subset of relation types.

The constructed KG therefore serves as the semantic input to the five-phase RAHGRec pipeline summarized below.

In Phase 1, RAHGRec constructs a unified input space that includes projects, libraries, and auxiliary knowledge-graph entities (e.g., maintainers, licenses, descriptive metadata). Project–library adoption is represented as typed directed relations (with inverses), while semantic knowledge-graph triples are retained as relation-typed facts (also augmented with inverses). Importantly, semantic relationships are organized into a set of *roles*, producing semantically coherent subsets that will later induce role-specific propagation channels.

In Phase 2, the framework constructs a *role-aware hypergraph family* from the semantic relations. For each semantic role, RAHGRec forms hyperedges by anchoring on an entity and grouping all entities directly connected to it under that role. This yields parallel, role-isolated hypergraph structures represented by sparse incidence matrices with degree normalization. The outcome is a collection of normalized role-specific incidence operators that encode a higher-order semantic context without mixing unrelated relation types.

In Phase 3, RAHGRec learns entity representations through *dual-channel propagation*. A usage-driven adjacency channel propagates behavioral signals over project–library adoption edges, while role-specific hypergraph channels propagate semantic context through normalized incidence operators. A learnable *role gating* mechanism controls the relative influence of different semantic roles during propagation. The resulting message is fused using a bi-interaction aggregation function, producing refined embeddings at each propagation depth.

In Phase 4, the model aggregates multi-layer embeddings into a single multi-scale representation for each entity. Rather than compressing layer output through attention or averaging, RAHGRec concatenates embeddings from all propagation layers (including the initialization), thereby preserving both local behavioral cues and the longer-range semantic context. The concatenated embedding is ℓ_2 -normalized to ensure stable scoring in the final stage.

In Phase 5, RAHGRec ranks the candidate libraries for each project using an inner-product scoring function over the learned embeddings. Training optimizes a pairwise ranking objective that encourages libraries observed (or semantically coherent) for a project to score higher than sampled negatives that are neither adopted nor strongly supported by semantic links. This learning objective aims at optimizing recommendation quality through implicit feedback, while regularization avoids overfitting and parameter explosion.

Taken together, these steps constitute the RAHGRec pipeline, which is detailed in the following subsections. The subsequent subsections provide the mathematical description of each step along with the optimization process and experimental setting.

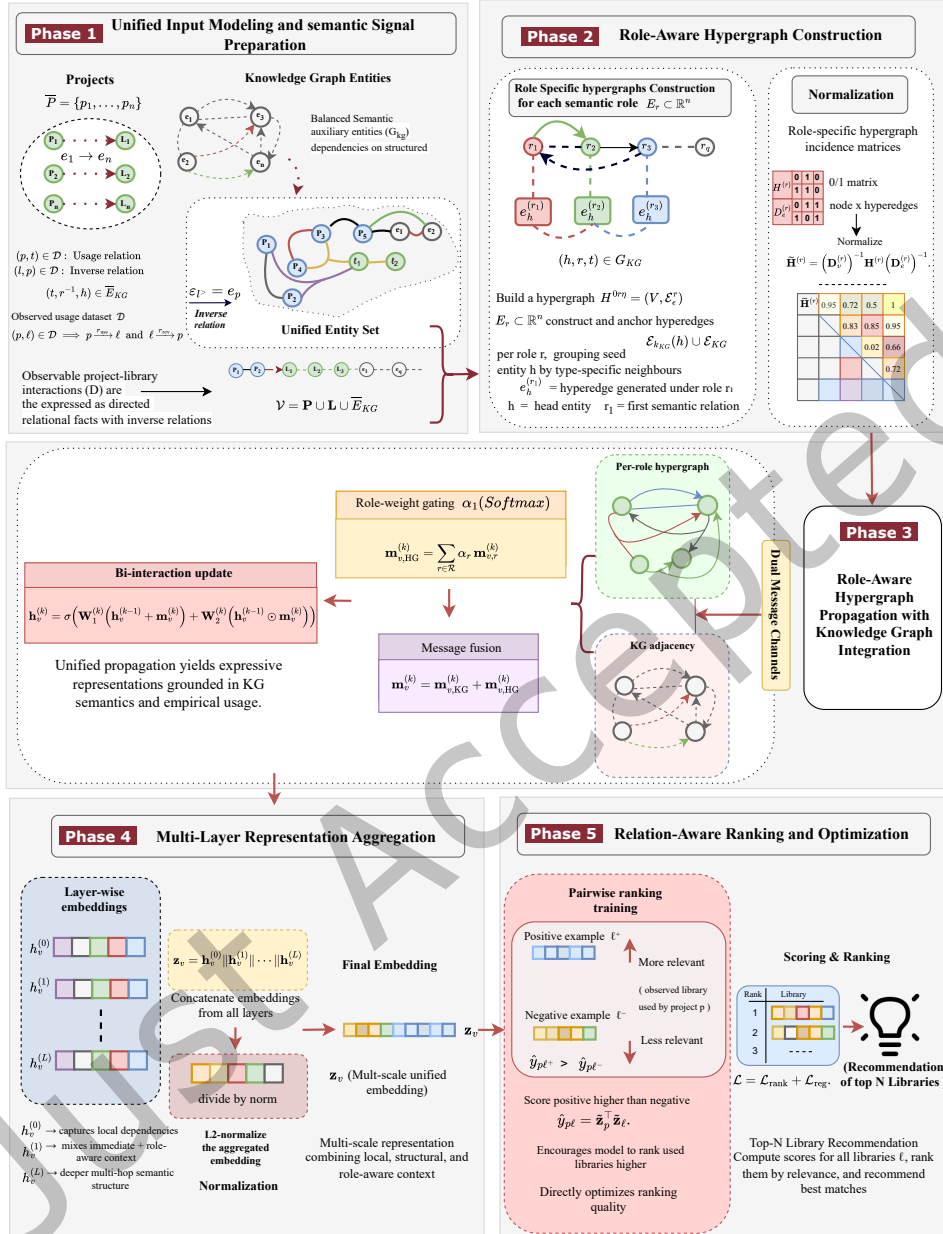


Fig. 3. General process flow of RAHGRec, including (1) unified input modeling with typed usage relations and role-isolated knowledge graph semantics, (2) construction of role-aware hypergraphs from semantic relations, (3) node representation learning via dual-channel usage adjacency and role-gated hypergraph propagation, (4) multi-layer embedding aggregation through concatenation, and (5) project-library recommendation via inner-product scoring and pairwise ranking loss.

Algorithm 1 RAHGRec: Role-Aware Hypergraph Modeling for Project-Library Recommendation with Knowledge Graph Semantics

Input: Implicit-feedback interactions $\mathcal{D}$; KG triples $\mathcal{G}_{\text{KG}}$; role set $\mathcal{R}$; embedding dimension d ; propagation layers L ; dropout rate p ; regularization coefficient λ ; top- K

Output: Top- K recommended project libraries for each project

- 1 **1. Unified input modeling and semantic signal preparation** Construct unified entity set $\mathcal{V} = \mathcal{P} \cup \mathcal{L} \cup \mathcal{E}_{\text{KG}}$ Augment usage with inverse relation: $\forall (p, \ell) \in \mathcal{D}$, add $(p, r_{\text{use}}, \ell)$ and $(\ell, r_{\text{rev}}, p)$ Augment KG with inverse triples: $\forall (h, r, t) \in \mathcal{G}_{\text{KG}}$, add (t, r^{-1}, h)
 - 2 Initialize entity embeddings and TransR-style KG parameters: $\forall v \in \mathcal{V}$, initialize $\mathbf{h}_v^{(0)}$ using Xavier uniform initialization $\forall r \in \mathcal{R}_{\text{KG}}$, initialize relation embedding $\mathbf{q}_r$ and projection matrix $\mathbf{M}_r$ using Xavier uniform initialization
 - 3 **2. Role-aware hypergraph construction (semantic roles only)** **foreach** $r \in \mathcal{R}$ **do**
 - 4 Construct role-specific hyperedges by anchoring on heads: $\forall h \in \mathcal{V}$, $e_h^{(r)} = \{h\} \cup \{t \mid (h, r, t) \in \mathcal{G}_{\text{KG}}\}$ Construct incidence $\mathbf{H}^{(r)} \in \{0, 1\}^{|\mathcal{V}| \times |\mathcal{E}^{(r)}|}$ with $\mathbf{H}_{v,e}^{(r)} = \mathbb{I}[v \in e]$ Compute degree matrices: $\mathbf{D}_v^{(r)}(v, v) = \sum_e \mathbf{H}_{v,e}^{(r)}$, $\mathbf{D}_e^{(r)}(e, e) = \sum_v \mathbf{H}_{v,e}^{(r)}$ Define normalized incidence operator: $\tilde{\mathbf{H}}^{(r)} = (\mathbf{D}_v^{(r)})^{-1} \mathbf{H}^{(r)} (\mathbf{D}_e^{(r)})^{-1}$
 - 5 **3. Dual-channel propagation (usage adjacency + role hypergraphs)** Construct usage-only normalized adjacency matrix $\mathbf{A}$ from $\{(p, r_{\text{use}}, \ell), (\ell, r_{\text{rev}}, p)\}$ Initialize global role-gating logits $\{\beta_r\}_{r \in \mathcal{R}}$ (shared across all layers)
 - 6 **for** $k = 1$ **to** L **do**
 - 7 Compute role weights: $\alpha_r = \frac{\exp(\beta_r)}{\sum_{r' \in \mathcal{R}} \exp(\beta_{r'})}$ for all $r \in \mathcal{R}$
 - 8 Stack embeddings into matrix $\mathbf{H}^{(k-1)} \in \mathbb{R}^{|\mathcal{V}| \times d}$ whose v -th row is $(\mathbf{h}_v^{(k-1)})^\top$
 - 9 **foreach** $v \in \mathcal{V}$ **do**
 - 10 Usage-adjacency message: $\mathbf{m}_{v,\text{use}}^{(k)} = \sum_{u \in \mathcal{N}_{\text{use}}(v)} \mathbf{A}_{vu} \mathbf{h}_u^{(k-1)}$
 - 11 Role-aware hypergraph message (semantic roles): $\mathbf{m}_{v,\text{hg}}^{(k)} = \sum_{r \in \mathcal{R}} \alpha_r \left(\tilde{\mathbf{H}}^{(r)} (\tilde{\mathbf{H}}^{(r)})^\top \mathbf{H}^{(k-1)} \right)_v$
 - 12 Fuse and update (bi-interaction): $\mathbf{m}_v^{(k)} = \mathbf{m}_{v,\text{use}}^{(k)} + \mathbf{m}_{v,\text{hg}}^{(k)}$ $\mathbf{h}_v^{(k)} = \sigma(\mathbf{W}_1^{(k)}(\mathbf{h}_v^{(k-1)} + \mathbf{m}_v^{(k)}) + \mathbf{W}_2^{(k)}(\mathbf{h}_v^{(k-1)} \odot \mathbf{m}_v^{(k)}))$ Apply dropout: $\mathbf{h}_v^{(k)} \leftarrow \text{Dropout}_p(\mathbf{h}_v^{(k)})$ Layer-wise ℓ_2 normalization: $\mathbf{h}_v^{(k)} \leftarrow \mathbf{h}_v^{(k)} / \|\mathbf{h}_v^{(k)}\|_2$
 - 13 **4. Multi-layer representation aggregation** **foreach** $v \in \mathcal{V}$ **do**
 - 14 Concatenate layer-wise embeddings: $\mathbf{z}_v = [\mathbf{h}_v^{(0)} \parallel \mathbf{h}_v^{(1)} \parallel \dots \parallel \mathbf{h}_v^{(L)}]$ Normalize for scoring: $\tilde{\mathbf{z}}_v = \mathbf{z}_v / \|\mathbf{z}_v\|_2$
 - 15 **5. Relation-aware ranking and optimization (implicit feedback)** **foreach** $project\ p \in \mathcal{P}$ **do**
 - 16 **foreach** *candidate library* $\ell \in \mathcal{L}$ *not used by* p **do**
 - 17 Compute relevance score: $\hat{y}_{p\ell} = \tilde{\mathbf{z}}_p^\top \tilde{\mathbf{z}}_\ell$
 - 18 Rank libraries by $\hat{y}_{p\ell}$ and return top- K
 - 19 Sample triplets (p, ℓ^+, ℓ^-) with $(p, \ell^+) \in \mathcal{D}$ and $\ell^- \notin \mathcal{D}(p)$ Optimize BPR loss: $\mathcal{L}_{\text{rank}} = \frac{1}{|\mathcal{T}|} \sum_{(p, \ell^+, \ell^-) \in \mathcal{T}} -\log \sigma(\hat{y}_{p\ell^+} - \hat{y}_{p\ell^-})$ Regularize *unnormalized* aggregated embeddings: $\mathcal{L}_{\text{reg}} = \frac{\lambda}{2} \sum_{v \in \mathcal{V}} \|\mathbf{z}_v\|_2^2$
 - 20 Minimize $\mathcal{L} = \mathcal{L}_{\text{rank}} + \mathcal{L}_{\text{KG}} + \mathcal{L}_{\text{reg}}$ using Adam in mini-batches (with early stopping on validation)
-

4.2 Phase 1: Unified Input Modeling and Semantic Signal Preparation

The proposed framework starts with the formalization of all the observable software artifacts and semantics in a unified relational space to ensure that high-level reasoning is performed on explicitly constructed structures. Let $\mathcal{P}$ denote the set of software projects, $\mathcal{L}$ the set of project libraries, and $\mathcal{E}_{\text{KG}}$ the set of auxiliary entities introduced by the knowledge graph, such as maintainers, licenses, and descriptive metadata. All elements are

embedded into a unified entity universe

$$\mathcal{V} = \mathcal{P} \cup \mathcal{L} \cup \mathcal{E}_{\text{KG}}, \quad (1)$$

where each entity $v \in \mathcal{V}$ is associated with an initial latent representation

$$\mathbf{h}_v^{(0)} \in \mathbb{R}^d. \quad (2)$$

To support typed KG relational learning, each relation r is also associated with a relation embedding $\mathbf{q}_r$ and a relation-specific projection matrix $\mathbf{M}_r$, following a TransR-style embedding formulation. Entity embeddings, relation embeddings, and projection matrices are initialized using Xavier uniform initialization [9],

$$\mathbf{h}_v^{(0)}, \mathbf{q}_r, \mathbf{M}_r \leftarrow \text{XavierUniform}(\cdot). \quad (3)$$

These parameters are optimized during later training through the recommendation and KG relational learning objectives. The relation-specific projection allows entities to participate differently under dependency, maintainer, license, or descriptive relations, while heterogeneous KG entities remain distinguished by the typed relations that later define role-specific propagation channels.

Observed software usage data is represented as a set of project–library interactions

$$\mathcal{D} = \{(p, \ell) \mid p \in \mathcal{P}, \ell \in \mathcal{L}\}, \quad (4)$$

indicating that project p adopts library ℓ . Instead of treating these interactions as untyped edges, each pair is modeled as a directed relational fact

$$(p \xrightarrow{r_{\text{use}}} \ell), \quad (5)$$

together with its inverse

$$(\ell \xrightarrow{r_{\text{rev}}} p), \quad (6)$$

allowing symmetric information flow while preserving the semantic asymmetry inherent to software usage relationships.

Complementary to interaction data, the knowledge graph provides structured semantic relations in the form of typed triples

$$\mathcal{G}_{\text{KG}} = \{(h, r, t) \mid h, t \in \mathcal{V}, r \in \mathcal{R}_{\text{KG}}\}, \quad (7)$$

where each relation r encodes a specific semantic association, such as authorship, dependency, or descriptive association. For consistency and balanced information flow during later propagation, each semantic relation is augmented with its inverse

$$(h, r, t) \Rightarrow (t, r^{-1}, h), \quad (8)$$

without collapsing or reweighting relation types at this stage.

Similarity-based semantic relations are constructed using Jaccard overlap between comparable feature sets: project topic sets for project–project similarity and library metadata sets for library–library similarity. A relation is retained only when the overlap satisfies its corresponding predefined threshold, with α_p used for project–project similarity and α_l used for library–library similarity. In our implementation, both thresholds are configurable hyperparameters and are set to 0.5 by default, meaning that a similarity relation is retained only when the compared entities share at least half of their combined feature evidence under the Jaccard criterion. Explicit relations such as library dependency and maintainer links are treated as typed KG triples directly, rather than inferred through Jaccard thresholding.

Crucially, semantic relations are not treated as a homogeneous signal. Instead, each relation type is treated as a distinct semantic role. Let

$$\mathcal{R}_{\text{role}} = \mathcal{R}_{\text{KG}} = \{r^{(1)}, r^{(2)}, \dots, r^{(K)}\} \quad (9)$$

denote the set of relation-defined roles, where each role corresponds to a unique functional interpretation (e.g., dependency-oriented, descriptive, or authorship-related semantics). Each role induces a role-specific relation subset

$$\mathcal{E}^{(k)} = \{(h, t) \mid (h, r^{(k)}, t) \in \mathcal{G}_{\text{KG}}\}, \quad (10)$$

thereby isolating homogeneous semantic signals and preventing interference across unrelated relation types.

At the end of this phase, all projects, libraries, and heterogeneous auxiliary KG entities reside in a shared latent embedding space $\mathcal{V}$, while their semantic differences are preserved through typed relations and decomposed into relation-defined roles. No propagation, aggregation, or role weighting is performed at this stage, Phase 1 only prepares the initialized embedding parameters and structured relational substrate used by the role-aware hypergraph construction in Phase 2.

4.3 Phase 2: Role-Aware Hypergraph Construction

Building on the semantically enriched, relation-typed interaction space created in Phase 1, the second phase develops a *role-aware hypergraph* that explicitly captures higher-order software relationships while retaining the functional roles encoded in the knowledge graph. Hypergraph-based recommendation has been widely used to model higher-order interactions beyond pairwise graph edges [3, 13, 42]. The main goal of this stage is to go beyond simple pairwise connections and build the richer, and more structured contexts that can connect several entities at once, while still preserving their individual roles instead of forcing everything into a single, and uniform graph.

Unified Node Space. Let $\mathcal{V}$ denote the combined collection of entities introduced in Phase 1, such as projects, libraries, and other elements of the knowledge-graph. We model each entity $v \in \mathcal{V}$ as a node in the hypergraph. At this stage, we refrain from imposing any artificial distinctions between nodes, instead, we capture the semantic differences through role-specific hyperedges rather than by assigning the distinct node types.

Role Decomposition of Relations. Let $\mathcal{R} = \{r_1, r_2, \dots, r_{|\mathcal{R}|}\}$ represent the set of relation types defined in knowledge graph. Building on Phase 1, we treat each type of the relation as its own *semantic role*. This approach allows us to track how different kinds of relations—such as dependency, authorship, or descriptive links—interact on their own, so they stay separate and are not blended together too soon.

Creating Hyperedges from Semantic Connections. For each semantic role $r \in \mathcal{R}$, we construct a role-specific hypergraph

$$\mathcal{H}^{(r)} = (\mathcal{V}, \mathcal{E}^{(r)}).$$

Each hyperedge $e_h^{(r)} \in \mathcal{E}^{(r)}$ is formed by anchoring a head entity h participating in the relation r and grouping all entities directly connected to h under that relation:

$$e_h^{(r)} = \{h\} \cup \{t \mid (h, r, t) \in \mathcal{G}_{\text{KG}}\}. \quad (11)$$

With this setup, each head entity participates in exactly one hyperedge per semantic role, so the total number of hyperedges scales linearly with the size of the knowledge graph.

Incidence Representation. For each role-specific hypergraph $\mathcal{H}^{(r)}$, we define an incidence matrix

$$\mathbf{H}^{(r)} \in \{0, 1\}^{|\mathcal{V}| \times |\mathcal{E}^{(r)}|}$$

such that

$$\mathbf{H}_{v,e}^{(r)} = \begin{cases} 1, & \text{if } v \in e^{(r)}, \\ 0, & \text{otherwise.} \end{cases} \quad (12)$$

This design preserves the complete membership pattern of each hyperedge while remaining sparse, which makes downstream propagation computationally efficient.

Degree Normalization. To allow information to propagate consistently across hyperedges of varying cardinality, we use a normalization scheme that accounts for both node degrees and hyperedge degrees. Let $\mathbf{D}_v^{(r)}$ and $\mathbf{D}_e^{(r)}$ denote the diagonal degree matrices of nodes and hyperedges, respectively:

$$\mathbf{D}_v^{(r)}(v, v) = \sum_e \mathbf{H}_{v,e}^{(r)}, \quad \mathbf{D}_e^{(r)}(e, e) = \sum_v \mathbf{H}_{v,e}^{(r)}. \quad (13)$$

The normalized incidence operator is defined as

$$\tilde{\mathbf{H}}^{(r)} = \left(\mathbf{D}_v^{(r)}\right)^{-1} \mathbf{H}^{(r)} \left(\mathbf{D}_e^{(r)}\right)^{-1}. \quad (14)$$

This normalization makes sure that entities with many connections or very large hyperedges don't unfairly overpower how messages are passed.

Multi-Role Hypergraph Structure. The complete role-aware hypergraph is depicted as a set of the parallel, and semantically distinct structures:

$$\mathbb{H} = \{\tilde{\mathbf{H}}^{(r)} \mid r \in \mathcal{R}\}. \quad (15)$$

Each $\tilde{\mathbf{H}}^{(r)}$ captures the richer interactions guided by a specific semantic role, allowing us to separately think about the dependency, authorship, and descriptive relationships.

Rationale and Phase Output. This formulation clearly distinguishes *structural co-participation* and *semantic causality*. Hyperedges capture richer, higher-order contextual links, while keeping roles distinct preserves the functional intent of each semantic relation. At this point, we do not perform any aggregation, weighting, or fusion across roles.

Phase 2 produces the set of normalized, role-specific incidence operators

$$\mathbb{H} = \{\tilde{\mathbf{H}}^{(r)}\}_{r \in \mathcal{R}},$$

which serves as the only structural input for the role-aware propagation mechanism introduced in Phase 3.

4.4 Phase 3: Role-Aware Hypergraph Propagation with Knowledge Graph Integration

After building the role-isolated hypergraph structures in Phase 2, the third phase focuses on learning expressive representations of projects and libraries through carefully managed information flow. This phase primarily aims to spread higher-order contextual cues across the ecosystem, while preserving the semantic cause-and-effect captured by different roles and maintaining alignment with the underlying knowledge graph relations.

Initial Embedding Space. Each entity $v \in \mathcal{V}$ is assigned a trainable embedding vector

$$\mathbf{h}_v^{(0)} \in \mathbb{R}^d, \quad (16)$$

Here, d denotes the dimensionality of the embeddings. As outlined in Phase 1, these embeddings act as the starting point for learning and stay fully trainable throughout all the following propagation layers.

Dual-Channel Message Propagation. At each propagation layer k , we refresh the entity representations by combining information from two complementary sources: (i) pairwise project–library usage-based adjacency propagation, and (ii) higher-order semantic context modeled with role-aware hypergraphs. This design allows us to treat behavioral interaction cues and semantic relationships separately, yet still fuse them in a controlled and consistent manner.

Usage-Based Adjacency Propagation. Let $\mathbf{A}$ denote the normalized adjacency matrix built exclusively from project–library usage relationships and their inverses, as described in Phase 1.

Following the neighborhood aggregation principle used in graph neural recommendation models [16, 20], the usage-based message received by entity v at the propagation layer k is calculated as

$$\mathbf{m}_{v,\text{use}}^{(k)} = \sum_{u \in \mathcal{N}_{\text{use}}(v)} \mathbf{A}_{vu} \mathbf{h}_u^{(k-1)}, \quad (17)$$

where $\mathcal{N}_{\text{use}}(v)$ denotes the set of entities directly connected to v through the adoption relations observed between the project and the library. This channel propagates fine-grained behavioral signals derived solely from empirical co-usage evidence, without introducing semantic abstraction.

Role-Aware Hypergraph Propagation. For each semantic role $r \in \mathcal{R}$, higher-order information is propagated through the corresponding normalized incidence operator $\tilde{\mathbf{H}}^{(r)}$ constructed in Phase 2, following standard hypergraph message-passing practice [3, 13, 42]. In RAHGRec, this propagation is performed separately for each role, using only the knowledge graph relations associated with that role. Propagation proceeds in two stages. First, hyperedge-level representations are computed by aggregating the embeddings of member entities:

$$\mathbf{z}_e^{(k,r)} = \sum_{v \in e} (\tilde{\mathbf{H}}^{(r)})_{v,e} \mathbf{h}_v^{(k-1)}, \quad (18)$$

for each hyperedge $e \in \mathcal{E}^{(r)}$. Second, node-level updates are obtained by redistributing hyperedge signals back to participating entities:

$$\mathbf{m}_{v,r}^{(k)} = \sum_{e \in \mathcal{E}^{(r)}} (\tilde{\mathbf{H}}^{(r)})_{v,e} \mathbf{z}_e^{(k,r)}. \quad (19)$$

Role-Level Gating Mechanism. Not all semantic roles contribute equally to the recommendation task. To enable adaptive regulation of semantic influence, a learnable gating mechanism assigns a scalar importance weight α_r to each role:

$$\alpha_r = \frac{\exp(\beta_r)}{\sum_{r' \in \mathcal{R}} \exp(\beta_{r'})}, \quad (20)$$

where β_r is a logit trainable parameter shared globally across all propagation layers to provide a stable and parsimonious role weighting. The aggregated semantic hypergraph message is then computed as

$$\mathbf{m}_{v,\text{HG}}^{(k)} = \sum_{r \in \mathcal{R}} \alpha_r \mathbf{m}_{v,r}^{(k)}. \quad (21)$$

Unified Message Fusion. The final message received by entity v in the propagation layer k is obtained by combining behavioral usage signals and the semantic context aware of the role:

$$\mathbf{m}_v^{(k)} = \mathbf{m}_{v,\text{use}}^{(k)} + \mathbf{m}_{v,\text{HG}}^{(k)}. \quad (22)$$

To enhance expressiveness and mitigate information loss, we use a bi-interaction aggregation strategy inspired by graph collaborative filtering [35]:

$$\mathbf{h}_v^{(k)} = \sigma(\mathbf{W}_1^{(k)}(\mathbf{h}_v^{(k-1)} + \mathbf{m}_v^{(k)}) + \mathbf{W}_2^{(k)}(\mathbf{h}_v^{(k-1)} \odot \mathbf{m}_v^{(k)})), \quad (23)$$

where $\mathbf{W}_1^{(k)}$ and $\mathbf{W}_2^{(k)}$ are trainable transformation matrices, $\odot$ denotes element-wise multiplication and $\sigma(\cdot)$ is implemented using the LeakyReLU activation function.

Stability Enhancements. To ensure stable training and prevent representation collapse, dropout is applied to intermediate representations, followed by ℓ_2 normalization of entity embeddings at each propagation layer. This stabilization is particularly important given the varied nature of role-aware hypergraph propagation and the presence of more complex, higher-order semantic structures.

Phase Output. After passing through the propagation layers L , each entity v is linked to a hierarchical set of representations

$$\{\mathbf{h}_v^{(0)}, \mathbf{h}_v^{(1)}, \dots, \mathbf{h}_v^{(L)}\}, \quad (24)$$

which together will represent the practical usage of the system, the semantic context involved, and the effects associated with certain roles. It is the only output produced in Phase 3, and it acts as the input for the multi-layer aggregation process mentioned in Phase 4.

4.5 Phase 4: Multi-Layer Representation Aggregation

During Phase 3, the propagation step, with consideration of the roles, produces stacks of embeddings for each entity, where each subsequent level represents more general structures and semantic contexts. While deep levels encode long-range dependencies and interactions, the shallow ones provide information about closer dependencies, such as direct usage relations and nearby semantic relationships. The goal of this phase is to fuse these complementary views into a single, reliable embedding that can be effectively applied to ranking and optimization.

Layer-Wise Embedding Collection. For each entity $v \in \mathcal{V}$, the propagation process produces a sequence of representations

$$\{\mathbf{h}_v^{(0)}, \mathbf{h}_v^{(1)}, \dots, \mathbf{h}_v^{(L)}\}, \quad (25)$$

Here, $\mathbf{h}_v^{(0)}$ denotes the initial embedding obtained in Phase 1, and $\mathbf{h}_v^{(k)}$ denotes the output of the k -th role-aware propagation layer. Each embedding captures information from a distinct receptive field shaped by project-library usage patterns and role-specific semantic hypergraphs.

Multi-Scale Feature Preservation. Instead of merging layer-wise representations via averaging or attention-based fusion, we preserve the signal from each propagation depth by concatenating the embeddings from all layers end-to-end, as follows:

$$\mathbf{z}_v = \mathbf{h}_v^{(0)} \parallel \mathbf{h}_v^{(1)} \parallel \dots \parallel \mathbf{h}_v^{(L)}, \quad (26)$$

In the above equation, $\parallel$ refers to vector concatenation. Such a straightforward stacking mechanism ensures that both fine and coarse information is retained in one place, which the following ranking function can readily leverage.

Rationale for Concatenation. Using multiple propagation depths is important in practice as each layer represents evidence of a certain kind. In shallow layers, there is an emphasis on strong local evidence such as direct connections, dependencies, and co-occurrences. In deeper layers, there is weak but meaningful global evidence such as indirect semantics and common context. Concatenation keeps the two perspectives without adding new trainable parameters to the model. This approach draws on previous research in the graph-based recommendation, which found that multi-layer representations contain complementary information and not mere the duplications of the same signal [13, 35].

Normalization for Scoring Stability. To ensure the stability of the relevance score and prevent any propagation depth to have undue influence on the others, we normalize the resultant embedding before deriving the score:

$$\tilde{\mathbf{z}}_v = \frac{\mathbf{z}_v}{\|\mathbf{z}_v\|_2}. \quad (27)$$

This normalized embedding is used solely for measuring similarity as part of the ranking process. Most importantly, the original embedding $\mathbf{z}_v$ is kept in its unnormalized form for parameter regularization during training, which prevents the normalization step from interfering with the overall learning process.

Phase Output. At the end of this phase, each project and library node will have an embedding $\mathbf{z}_v$ and a normalized one $\tilde{\mathbf{z}}_v$. Each of these embeddings will be informed by multiple hop structures, role-based semantics, interactions, and will form the only input into the relation-aware ranking and optimization phase of Phase 5.

4.6 Phase 5: Relation-Aware Ranking and Optimization

Based on the unified multi-scale representations established for each project and library in Phase 4, the final phase is concerned with assessing relevance, learning ranking functions to separate good suggestions from poor ones, and adjusting the model parameters. The objective is to highlight libraries that are both structurally compatible and semantically similar to a particular project, filtering out those that are not relevant or only remotely related.

Relevance Scoring. Let $\tilde{\mathbf{z}}_p$ and $\tilde{\mathbf{z}}_\ell$ be the normalized embeddings for project p and library ℓ , respectively. Following the standard latent-factor recommendation practice [18], we compute the relevance of a given project and the library pair using their inner product.

$$\hat{y}_{p\ell} = \tilde{\mathbf{z}}_p^\top \tilde{\mathbf{z}}_\ell. \quad (28)$$

This design is computationally efficient and offers an intuitive view of how entities align within the shared embedding space. Because semantic roles and knowledge graph cues are already incorporated during propagation, higher scores naturally indicate libraries that are both commonly co-adopted and semantically well-aligned with the project context.

Pairwise Ranking Objective. To learn relative preference rankings from implicit feedback, the model is trained using a Bayesian personalized ranking objective [30]. For each observed project-library pair (p, ℓ^+) , we sample a negative library ℓ^- that is neither used by p nor closely related through semantic connections. The training objective encourages the model to give a higher relevance score to ℓ^+ than to ℓ^- :

$$\mathcal{L}_{\text{rank}} = \frac{1}{|\mathcal{T}|} \sum_{(p, \ell^+, \ell^-) \in \mathcal{T}} -\log \sigma(\hat{y}_{p\ell^+} - \hat{y}_{p\ell^-}). \quad (29)$$

Here, $\sigma(\cdot)$ denotes the sigmoid function, and $\mathcal{T}$ is the set of sampled training triplets. This objective is well-suited to implicit feedback settings and aims to improve relative ranking performance rather than the absolute values of the scores.

In addition to the recommendation loss, RAHGREC optimizes a KG relational learning loss over typed semantic triples using the TransR-style relation embedding and projection parameters introduced in Phase 1. For each positive KG triple $(h, r, t) \in \mathcal{G}_{\text{KG}}$, we define the TransR-style score as

$$s_r(h, t) = \|\mathbf{M}_r \mathbf{h}_h + \mathbf{q}_r - \mathbf{M}_r \mathbf{h}_t\|_2^2,$$

where $\mathbf{q}_r$ is the embedding of relation r , and $\mathbf{M}_r$ is the relation-specific projection matrix. For each positive triple, we generate corrupted triples by replacing either the head or the tail entity with a sampled type-compatible

entity that does not form an observed positive triple. Let $\mathcal{G}_{\text{KG}}^-$ denote the resulting set of corrupted triples. The auxiliary KG objective is defined as

$$\mathcal{L}_{\text{KG}} = \sum_{(h,r,t) \in \mathcal{G}_{\text{KG}}} \sum_{(h',r,t') \in \mathcal{G}_{\text{KG}}^-} [\gamma + s_r(h,t) - s_r(h',t')]_+,$$

where $[\cdot]_+ = \max(0, \cdot)$ and γ is the margin. This objective encourages valid KG triples to obtain lower scores than corrupted triples, thereby preserving typed KG structure during representation learning.

Regularization. To mitigate overfitting and limit the magnitude of the embeddings, we apply ℓ_2 regularization to the *unnormalized* aggregated embeddings obtained in Phase 4. More precisely, let $\mathbf{z}_v$ represent the concatenated multi-layer embedding before normalization is performed. The regularization term is defined as

$$\mathcal{L}_{\text{reg}} = \frac{\lambda}{2} \sum_{v \in \mathcal{V}} \|\mathbf{z}_v\|_2^2, \quad (30)$$

Here, λ is a tunable regularization weight. With this design, regularization reliably keeps the representation from growing too large, while the normalized embeddings used for scoring remain unchanged.

The complete training objective is given by

$$\mathcal{L} = \mathcal{L}_{\text{rank}} + \mathcal{L}_{\text{KG}} + \mathcal{L}_{\text{reg}}. \quad (31)$$

Optimization Strategy. We train all parameters of the model, including entity embeddings, relation embeddings, relation-specific projection matrices, transformation matrices, and role-gating coefficients, using mini-batch optimization with the Adam optimizer [15]. This allows the model to converge well on large-scale recommendation tasks and also enables a smooth combination of the various signals provided by role-aware hypergraph propagation and knowledge graph relationships. Training will continue until convergence or early stopping based on validation performance.

Inference. During inference, we compute the relevance of each candidate library ℓ to a project p by computing relevance scores $\hat{y}_{p\ell}$ for all candidate libraries that have not yet been linked to project p . We then rank the libraries by relevance score in descending order and return the top k results. This step is computationally efficient, even when the number of candidate libraries is very large, since it only involves simple vector dot products.

It is at this point that the structured role-informed representations developed earlier become actual recommendations. Rather than making predictions one library at a time, the model focuses on ranking libraries, which enables it to move beyond the usage patterns and point to semantically similar libraries that may not be in the original data. This final ranking step completes the end-to-end role-aware hypergraph modeling framework for project-library recommendation.

5 Experimental Evaluation

This section presents a detailed empirical evaluation of RAHGRec. Our goal is to assess its effectiveness for project library recommendation under an implicit-feedback setting, and to examine whether it benefits from modeling higher-order structural dependencies together with role-specific semantics. The assessment will be performed not only by assessing the overall correctness of the recommendation but also by testing the stability of the suggested framework across various architectural options, different settings of hyperparameters, and varying scales. The proposed framework will be compared against some competitors which are representative of the current state-of-the-art using a pre-defined set of research questions.

5.1 Experimental Setup

The experimental design is organized around six research questions, with each research question representing an assumption that supports the methodology under consideration. In total, the questions evaluate the effectiveness, stability, and generalizability of the proposed algorithm in different settings.

- **RQ1:** Does RAHGREC improve library recommendation by jointly modeling higher-order co-adoption patterns and typed semantic relations compared with structure-only and semantics-only methods?
- **RQ2:** How does the depth of role-aware hypergraph propagation affect representation quality and recommendation accuracy, and when do over-smoothing or semantic dilution occur?
- **RQ3:** How do similarity thresholds and role-specific weights regulate higher-order information propagation and improve contextual compatibility in library recommendation?
- **RQ4:** How does latent embedding dimensionality affect RAHGREC’s ability to capture multi-scale structural and semantic signals without overfitting?
- **RQ5:** How robust and scalable is RAHGREC under larger datasets, sparse interactions, and long-tail project distributions, while maintaining computational efficiency?
- **RQ6:** How do the major components of RAHGREC contribute to recommendation performance?

To answer these questions, we conduct an extensive experimental evaluation involving RAHGREC as well as a variety of different cutting-edge library recommendation algorithms, such as collaborative filtering, matrix factorization, graph neural networks, and hypergraphs. The purpose of such a comparison is to distinguish the impact of role-aware hypergraph modeling and controlled semantic diffusion and examine its sensitivity to various practical constraints, including data sparsity, heterogeneous relation types, and computational budgets, influence performance.

All experiments are implemented in PyTorch. We run our evaluation on a workstation with an Intel® Core™ i7 CPU, 32 GB DDR4 memory, and an NVIDIA® RTX A5000 GPU. The software environment includes Ubuntu, PyTorch 2.2.1+cu121, NumPy 1.26.4, SciPy 1.11.4, CUDA 12.2, and NVIDIA driver 535.230.02.

We evaluate RAHGREC using the processed Python project–library benchmark included in our consolidated replication package³. The benchmark is derived from the PyRec dataset [21] and is designed for studying recommendation methods that combine structural project–library usage signals with semantic context. It contains 12,421 Python projects and 963 distinct libraries, with 121,474 project–library interactions derived from import relationships in GitHub repositories and associated PyPI metadata. In addition to usage relationships, the dataset contains semantic information such as project tags, library dependencies, and authors, with 73,277 semantic relationships and 9,675 auxiliary entities. As is common practice in the literature, we filter out projects associated with fewer than five libraries to provide sufficient semantic context for a meaningful evaluation.

To better model development environments where projects incrementally learn to use new libraries, we propose a masked cross-validation protocol in an implicit feedback setting. For each project, a random set of the project’s corresponding libraries is masked, and the rest of the interactions are used for training. The model is then challenged to predict the hidden libraries. The removal ratio r_m is set to $\{20\%, 40\%, 60\%\}$ to model varying levels of project maturity and sparsity of interaction. The quality of recommendations is measured for list sizes $K \in \{5, 10, 20\}$.

5.1.1 Evaluation Metrics. We evaluate recommendation as a top- K implicit-feedback ranking task. Each project p in the query-project set $\mathcal{Q}$ is treated as a query. For each query project, the masked libraries form the ground-truth set $\mathcal{G}_p$, while the remaining observed libraries are used as training evidence. Candidate libraries are selected from libraries that are not retained in the training interactions of p . The model ranks these candidates by predicted relevance and returns the top- K list $\mathcal{R}_p^K$. A recommendation is counted as relevant if it appears in $\mathcal{G}_p$.

For each project p , Precision@ K [28] measures the fraction of recommended libraries that are relevant:

$$P@K(p) = \frac{|\mathcal{R}_p^K \cap \mathcal{G}_p|}{K}. \quad (32)$$

Recall@ K [32] measures the fraction of withheld relevant libraries that are recovered:

$$R@K(p) = \frac{|\mathcal{R}_p^K \cap \mathcal{G}_p|}{|\mathcal{G}_p|}. \quad (33)$$

F-score@ K [25] is computed as the harmonic mean of Precision@ K and Recall@ K :

$$F@K(p) = \frac{2 \cdot P@K(p) \cdot R@K(p)}{P@K(p) + R@K(p)}. \quad (34)$$

When both precision and recall are zero, F-score@ K is set to zero.

Reciprocal Rank@ K [24] measures the position of the first relevant recommendation:

$$RR@K(p) = \begin{cases} \frac{1}{\text{rank}_p}, & \text{if } \mathcal{R}_p^K \cap \mathcal{G}_p \neq \emptyset, \\ 0, & \text{otherwise,} \end{cases} \quad (35)$$

where rank_p is the rank position of the first recommended library that belongs to $\mathcal{G}_p$.

The reported Mean Precision (MP), Mean Recall (MR), Mean F-score (MF), and Mean Reciprocal Rank (MRR) are obtained by averaging the corresponding per-project values over all query projects $\mathcal{Q}$:

$$MP@K = \frac{1}{|\mathcal{Q}|} \sum_{p \in \mathcal{Q}} P@K(p), \quad MR@K = \frac{1}{|\mathcal{Q}|} \sum_{p \in \mathcal{Q}} R@K(p). \quad (36)$$

$$MF@K = \frac{1}{|\mathcal{Q}|} \sum_{p \in \mathcal{Q}} F@K(p), \quad MRR@K = \frac{1}{|\mathcal{Q}|} \sum_{p \in \mathcal{Q}} RR@K(p). \quad (37)$$

Coverage (COV) [20] is used as a diversity-oriented metric and measures the fraction of distinct libraries that appear in at least one recommendation list across all query projects, relative to the full candidate library set. Together, these metrics evaluate recommendation accuracy, ranking quality, and diversity under the same masked evaluation protocol.

To enhance statistical reliability, we repeat each experimental setting 50 times with different random splits and report the mean performance. All baseline methods are assessed using the identical data splits, evaluation metrics, and stopping criteria, thereby supporting a fair and well-controlled comparison. We evaluate the statistical significance of the performance differences using non-parametric paired tests with a significance level of $p < 0.05$.

5.2 RQ1: Does RAHGRec improve library recommendation by jointly modeling higher-order co-adoption patterns and typed semantic relations compared with structure-only and semantics-only methods?

In order to evaluate the effectiveness of the proposed approach, we conducted an extensive comparative analysis of RAHGRec against various state-of-the-art methods for recommending project libraries. We considered several baselines from different modeling approaches, such as collaborative filtering, matrix factorization, graph neural networks (GNN), and hypergraph neural networks (HGNN). The aim of this comparison is to verify whether the joint modeling of higher-order structural properties along with role-based semantics leads to a statistically significant improvement in recommendation performance.

These baselines are based on well-established and highly competitive methods described in the literature:

Table 3. Performance comparison of RAHGRec and baseline library recommendation approaches under different sparsity levels ($r_m \in \{20\%, 40\%, 60\%\}$) and top- K recommendation list sizes ($K \in \{5, 10, 20\}$). Best results for each setting are highlighted in bold.

Dataset Approach		K = 5					K = 10					K = 20				
		MP	MR	MF	MRR	COV	MP	MR	MF	MRR	COV	MP	MR	MF	MRR	COV
$r_m = 20\%$	LibRec [32]	0.0819	0.2362	0.1216	0.2682	0.2669	0.0607	0.3731	0.0980	0.2850	0.2735	0.0332	0.3907	0.0583	0.3162	0.3089
	CrossRec [27]	0.0305	0.1136	0.0457	0.0723	0.1268	0.0262	0.1888	0.0442	0.0847	0.1458	0.0231	0.3333	0.0421	0.0964	0.1549
	LibSeek [11]	0.0985	0.3318	0.1418	0.2952	0.2847	0.0653	0.4291	0.1073	0.3092	0.3297	0.0413	0.5317	0.0739	0.3163	0.3765
	GRec [20]	0.0968	0.3394	0.1418	0.2926	0.3836	0.0645	0.4328	0.1074	0.3079	0.4593	0.0412	0.5390	0.0741	0.3153	0.5542
	HCF [3]	0.1082	0.3595	0.1604	0.3091	0.6191	0.0722	0.4829	0.1248	0.3385	0.5942	0.0458	0.5902	0.0833	0.3431	0.5792
	PyRec [21]	0.1106	0.3781	0.1711	0.3276	0.3857	0.0724	0.4844	0.1259	0.3424	0.4609	0.0455	0.5926	0.0845	0.3495	0.5553
	MsRec [13]	0.1098	0.3720	0.1688	0.3220	0.6303	0.0723	0.4836	0.1254	0.3405	0.6012	0.0456	0.5915	0.0868	0.3460	0.6220
	RAHGRec	0.12510.42960.19380.38020.5813	0.08020.53610.13950.39410.6134	0.04950.64010.09190.40060.6567												
$r_m = 40\%$	LibRec [32]	0.1497	0.2297	0.1672	0.3883	0.2778	0.0907	0.2616	0.1240	0.3932	0.2867	0.0497	0.2716	0.0781	0.3936	0.3186
	CrossRec [27]	0.0528	0.0912	0.0631	0.1279	0.1547	0.0522	0.1738	0.0759	0.1481	0.1965	0.0487	0.3286	0.0816	0.1651	0.2144
	LibSeek [11]	0.1675	0.2654	0.1907	0.4203	0.2979	0.1169	0.3598	0.1646	0.4359	0.3488	0.0774	0.4647	0.1256	0.4427	0.3915
	GRec [20]	0.1089	0.1928	0.1321	0.2680	0.4164	0.0804	0.2755	0.1184	0.2855	0.4968	0.0580	0.3798	0.0965	0.2951	0.5834
	HCF [3]	0.2049	0.3351	0.2537	0.5062	0.5133	0.1415	0.4402	0.2135	0.5221	0.4956	0.0913	0.5538	0.1574	0.5283	0.5774
	PyRec [21]	0.2111	0.3438	0.2616	0.5174	0.4215	0.1438	0.4549	0.2184	0.5312	0.4994	0.0926	0.5679	0.1591	0.5366	0.5852
	MsRec [13]	0.2080	0.3400	0.2580	0.5120	0.5280	0.1425	0.4470	0.2202	0.5260	0.4950	0.0975	0.5670	0.1580	0.5310	0.5789
	RAHGRec	0.24120.39210.30780.57780.5401	0.16130.50130.24180.59170.5132	0.10230.61040.17960.59210.5931												
$r_m = 60\%$	LibRec [32]	0.1246	0.1226	0.1138	0.3249	0.3279	0.0731	0.1353	0.0862	0.3268	0.3374	0.0396	0.1388	0.0561	0.3269	0.3443
	CrossRec [27]	0.0680	0.0738	0.0667	0.1290	0.2488	0.0880	0.1893	0.1133	0.1611	0.3032	0.0731	0.3245	0.1145	0.1768	0.3906
	LibSeek [11]	0.1705	0.1801	0.1643	0.3925	0.3507	0.1318	0.2715	0.1660	0.4094	0.4041	0.0887	0.3651	0.1340	0.4148	0.4219
	GRec [20]	0.1401	0.1578	0.1410	0.3178	0.5053	0.1039	0.2271	0.1350	0.3344	0.5721	0.0799	0.3154	0.0965	0.4004	0.6458
	HCF [3]	0.2781	0.2984	0.2883	0.5995	0.6264	0.1883	0.4052	0.2668	0.6121	0.5961	0.1302	0.5083	0.2089	0.6183	0.6703
	PyRec [21]	0.2862	0.3063	0.2959	0.6122	0.5331	0.2014	0.4196	0.2721	0.6229	0.6043	0.1320	0.5324	0.2116	0.6265	0.6781
	MsRec [13]	0.2845	0.3030	0.2920	0.6075	0.6349	0.1980	0.4150	0.2700	0.6200	0.6010	0.1315	0.5180	0.2105	0.6240	0.6750
	RAHGRec	0.31460.33800.32640.65610.6631	0.22030.45160.29640.66510.6310	0.14100.56030.22780.66780.6990												

- **LibRec** [32] is one of the first library recommendation algorithms that combines the principles of collaborative filtering and association rule mining to reveal frequently co-used libraries, and serves as an essential baseline for further studies.
- **CrossRec** [27] improves the performance of collaborative filtering by leveraging the relations between projects, thus improving trust in the results for projects with little interaction history.
- **LibSeek** [11] employs matrix factorization to reduce popularity bias and enhance recommendation diversity, with a primary focus on Android applications.
- **GRec** [20] models project–library relationships as a bipartite graph and leverages GNNs to capture multi-hop relational dependencies.
- **HCF** [3] leverages hypergraph structures to capture higher-order co-usage patterns, yet handles all relations in a uniform way without accounting for their semantic differences.
- **PyRec** [21] enhances graph-based recommendation with an added knowledge graph to capture semantic cues such as dependencies and topics, yet still only models pairwise interactions.
- **MsRec** [13] employs a multiscale hypergraph neural network to model co-usage patterns at different levels of granularity, but it overlooks semantic relations and treats all interactions as equally important.

Table 4. Statistical comparison based on Table 3. The results include a Friedman test across all algorithms and pairwise Wilcoxon signed-rank tests comparing RAHGRec with each baseline. The Wilcoxon results are BH-corrected, and rank-biserial correlation (r_{rb}) is reported as the effect size.

Test / Comparison	Effect Size	p -value	Significant
Friedman Test (on block-wise average ranks)			
Overall (all algorithms)	Kendall's $W = 0.9744$	7.98×10^{-11}	Yes
Wilcoxon Signed-Rank (BH-corrected, RAHGRec vs)			
LibRec	$r_{rb} = 1.00$	3.906×10^{-3}	Yes
CrossRec	$r_{rb} = 1.00$	3.906×10^{-3}	Yes
LibSeek	$r_{rb} = 1.00$	3.906×10^{-3}	Yes
GRec	$r_{rb} = 1.00$	3.906×10^{-3}	Yes
HCF	$r_{rb} = 1.00$	3.906×10^{-3}	Yes
PyRec	$r_{rb} = 1.00$	3.906×10^{-3}	Yes
MsRec	$r_{rb} = 1.00$	3.906×10^{-3}	Yes

For a fair and reproducible comparison, all baseline approaches are evaluated using the original parameter configurations reported in their respective studies [3, 11, 13, 20, 21, 27, 32]. For RAHGRec, we set the embedding dimensionality to $d = 128$ and employ $L = 2$ propagation layers, while all remaining hyperparameters are selected by validation on held-out data. This configuration is chosen to balance representational expressiveness and computational cost. To ensure a fair comparison, all models are trained and evaluated using the same data splits, sparsity settings, and experimental protocol. We also report the results using a common set of evaluation metrics.

Table 3 summarizes the results for all methods using mean precision (MP), mean recall (MR), mean F1 (MF), mean reciprocal rank (MRR), and coverage (COV), across multiple sparsity levels and the recommendation cutoffs K . In general, the proposed approach performs the best on every reported metric across all configurations. Notably, this advantage holds as sparsity increases, suggesting that the proposed framework remains effective even when historical library-adoption evidence is scarce. Compared with structure-focused methods such as GRec and MsRec, and semantics-oriented methods such as PyRec, the results indicate that jointly modeling higher-order co-adoption structure and role-aware semantic relations yields stronger recommendation quality than modeling either component in isolation.

Although some absolute values in Table 3 appear modest, they should be interpreted in light of the difficulty of the evaluation setting. Project–library recommendation is evaluated here as a sparse implicit-feedback top- K ranking problem, where each project has only a limited set of observed library adoptions and a subset of these adoptions is withheld as ground truth. Under this protocol, the number of relevant withheld libraries for a query can be small, while the model must rank candidates from a much larger library space. As a result, precision values can appear modest, particularly for larger K , because the denominator of precision increases with the recommendation list length and unobserved libraries are treated as non-relevant even if they may be practically useful. Therefore, the absolute values should not be interpreted as direct evidence of deployment-level utility. Rather, they provide controlled comparative evidence about how different methods behave under the same sparse benchmark setting.

Under moderate sparsity ($r_m = 20\%$), RAHGRec delivers substantial gains over classical collaborative-filtering baselines. For example, compared to LibRec, it improves MP, MR, MF, MRR, and COV by 52.7%, 81.9%, 59.4%, 41.8%, and 117.8%, respectively. The gap widens further when compared with CrossRec, where several metrics improve by more than 300%. RAHGRec also outperforms stronger neural and hypergraph-based baselines such as GRec and LibSeek, which supports the value of modeling higher-order structural dependencies alongside semantic context.

The benefit of the proposed framework becomes more pronounced as the interaction data grow sparser. Even at $r_m = 60\%$ sparsity, the proposed approach remains ahead of the classical collaborative methods and preserves a clear gap over recent neural and hypergraph-based baselines. Across MP, MR, MF, and MRR, its relative gains over HCF, PyRec, and MsRec fall in the range of roughly 7%-13%. This pattern indicates that role-aware semantic modeling and higher-order structural reasoning reinforce each other, particularly in sparse settings where both behavioral evidence and semantic context are limited.

To test whether these improvements reflect a consistent advantage rather than random variation, we evaluate statistical significance using the following hypotheses:

- H_0 : *There is no statistically significant difference between RAHGRec and the baseline methods.*
- H_1 : *RAHGRec statistically outperforms the baseline methods.*

We conduct statistical tests on the configuration-level results reported in Table 3. Each experimental block represents a specific pairing of sparsity level ($r_m \in \{20\%, 40\%, 60\}$) and recommendation cutoff ($K \in \{5, 10, 20\}$). By using configurations as the unit of analysis, we emphasize robustness across experimental conditions, rather than variability driven by particular random initializations. This setup directly reflects whether a method remains reliable as sparsity and cutoff settings change.

We first apply the Friedman test [7] to test the block-wise average ranks of all competing approaches. As shown in Table 4, it can be seen that the test finds a significant difference among the approaches (Kendall's [14] $W = 0.9744$, $p = 7.98 \times 10^{-11}$). A large W value suggests that the ranking order of the approaches is consistent across different blocks, while a low p -value suggests rejecting the null hypothesis that all methods perform similarly.

We then carry out Wilcoxon signed-rank tests [36] to compare the proposed approach with each baseline using configuration-level blocks. We employ the Benjamini-Hochberg procedure [1] to control the false discovery rate and address the multiple testing problem. All comparisons remain statistically significant after correction ($p = 3.91 \times 10^{-3}$). All rank-biserial correlations are found to be $r_{rb} = 1.0$, which indicates that the proposed approach provides better results compared to each of the baselines in different configurations. In the case of dominance of one approach over others, rank tests might give identical signed-ranks for different comparisons. Hence, obtaining identical adjusted p -values and effect sizes is a natural phenomenon and not a numerical artifact.

Beyond the absolute metric values, Recall, MRR, and coverage provide complementary views of the results, recall measures how many withheld libraries are recovered, MRR reflects whether relevant libraries appear early in the ranked list, and the coverage indicates whether the method avoids recommending only a narrow set of popular libraries. The ablation study in Section 5.7 further supports the interpretation of the proposed design. Removing either the higher-order hypergraph component or the role-aware semantic propagation leads to clear performance drops, indicating that these components make complementary contributions. At the same time, practical deployment would require additional validation, such as developer-centered studies or online evaluation in real recommendation scenarios.

RAHGRec consistently outperforms structure-only, semantics-oriented, graph-based, and hypergraph-based baselines across all sparsity levels and top- K settings. The results indicate that jointly modeling higher-order co-adoption patterns and role-aware semantic relations provides more effective project-library recommendations than modeling either source of information in isolation.

5.3 RQ2: How does the depth of role-aware hypergraph propagation affect representation quality and recommendation accuracy, and when do over-smoothing or semantic dilution occur?

This research question is about the impact of propagation depth in the performance of RAHGRec algorithm in terms of the number of role-aware hypergraph propagation layers for learning project and library representations. The propagation depth factor is very important for graph and hypergraph recommenders since it regulates how deep the structural and semantic information can propagate through the graph. While more complex models may capture higher-order dependencies, excessive propagation risks over-smoothing and semantic dilution, particularly in heterogeneous graphs.

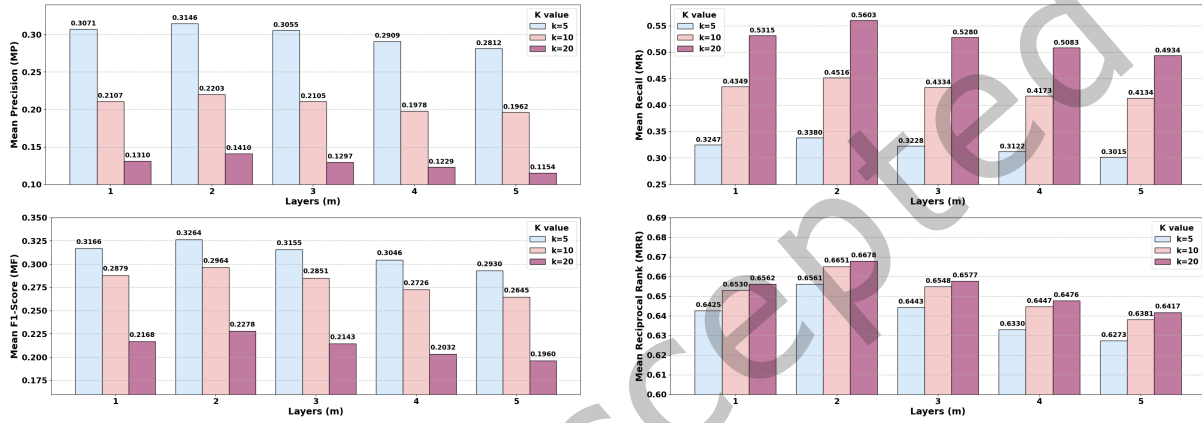


Fig. 4. Impact of the number of embedding propagation layers on RAHGRec performance metrics for $K = 5, K = 10$ and $K = 20$ (a) MP vs Number of Layers. (b) MR vs Number of Layers. (c) MF vs Number of Layers. (d) MRR vs Number of Layers.

To investigate the trade-off in more detail, we consider different numbers of propagation layers, ranging from one to five, and evaluate the performance of the model on a hard sparsity scenario ($r_m = 60\%$), in which many interaction pairs between projects and libraries are deliberately removed from the data. This sparsity setup requires the use of multi-hop reasoning because the signals provided by shallow models come mostly from direct usage, which is absent in the sparsity scenario considered. The quality of recommendations is evaluated using metrics such as mean precision (MP), mean recall (MR), mean F1 (MF), and mean reciprocal rank (MRR) at $K \in \{5, 10, 20\}$ points. The impact of model depth on performance is shown in Figure 4.

From the experimental results, we can observe that with the increase in depth from one layer to two layers of propagation, the metrics give better performances for all the recommended cut-offs as depicted in Figure 4. For MP, MR, MF, and MRR, the two-layer propagation setting with $K \in \{5, 10, 20\}$ achieves the best results, which clearly indicates that shallow multi-hop propagation helps RAHGRec better capture the higher-order co-usage patterns and role-aware semantics.

However, pushing the propagation depth past two layers leads to a slow but steady decline in recommendation performance. As the depth increases to three, four, and five layers, multiple evaluation metrics start to worsen. For example, MP at $K = 10$ decreases noticeably once the depth exceeds two layers, and MR and MRR follow the similar downward patterns, suggesting diminishing benefits of going deeper. This performance decline implies that additional layers introduce redundant or only weakly informative signals, making node representations more homogeneous and, therefore, less discriminative.

This can be explained by the over-smoothing effect caused by the repeated transmission of messages in the heterogeneous hyperedges. Although RAHGRec makes an explicit distinction between semantic roles and uses gated aggregation to reduce interference between roles, adding depth to the model still enhances noise and suppresses the signals related to the task. In particular, deeper layers can spread semantically distant information that offers little value to the project–library matching objective.

A two-layer propagation architecture provides the best trade-off between expressive capacity and representation robustness in RAHGRec. It captures useful higher-order structural and role-aware semantic information, while deeper propagation introduces over-smoothing and semantic dilution that reduce recommendation accuracy.

5.4 RQ3: How do similarity thresholds and role-specific weights regulate higher-order information propagation and improve contextual compatibility in library recommendation?

This research question investigates the sensitivity of RAHGRec to the key hyperparameters that govern the interaction between higher-order structural co-usage signals and semantic information derived from the software knowledge graph. Specifically, similarity thresholds α_p and α_l regulate which project-project and library-library relations are retained during hypergraph construction, while role-specific weighting factors λ_p and λ_l control the relative influence of semantic signals such as dependency relations, topical similarity, and maintainer information during message propagation. Together, these parameters determine how effectively semantic constraints regulate higher-order information flow and suppress semantically incompatible signals. Here, λ_p and λ_l denote semantic weighting factors used only in the sensitivity analysis, and are distinct from the regularization coefficient λ in the training objective.

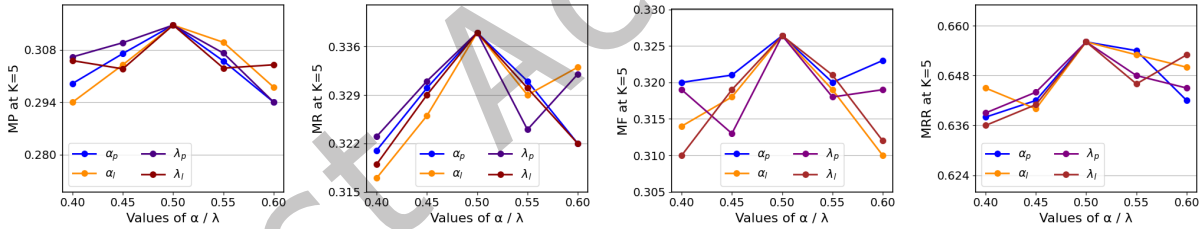


Fig. 5. Impact of similarity thresholds (α_p , α_l) and role-specific semantic weights (λ_p , λ_l) on RAHGRec performance under $r_m = 60\%$. Moderate values most effectively regulate higher-order propagation, while overly weak or overly strict constraints degrade recommendation quality.

To systematically analyze this behavior, we perform a controlled sensitivity analysis in a high-sparsity environment ($r_m = 60\%$), where the benefits of semantic regulation and higher-order modeling are most pronounced. Each parameter is independently varied within the range $[0.40, 0.60]$, while all remaining parameters are fixed at 0.50. Concretely, we first vary λ_p while fixing $\alpha_p = \alpha_l = \lambda_l = 0.50$, then vary λ_l under the same constraints. We repeat this procedure for α_p and α_l , allowing an isolated assessment of the contribution of each parameter under a consistent experimental baseline. Figure 5 summarizes how MP, MR, MF, and MRR change as we vary these parameters.

Across all metrics, there is an observable trend that is clear to see. An increase in the value of the parameters from 0.40 to 0.50 results in constant improvement. For example, at $K = 5$, there is an improvement in the mean

precision from 0.299 to 0.3146 as the value of α_p is set to 0.50. In addition, there is also an improvement in the mean recall from 0.327 to 0.3386 in the same range. MF and MRR follow the same trajectory, suggesting that moderate semantic constraints help suppress noisy or weakly related neighbors without discarding the structural and contextual evidence needed for accurate recommendation.

Beyond the balanced setting at 0.50, the performance degrades for all metrics. Both increasing the values of thresholds and semantic weight to 0.60 leads to a drop in MF (0.3264 to 0.310 with $K = 5$) and MRR (0.6561 to 0.632). This behavior is consistent with over-regularization: too restrictive constraints discard important structural information, whereas an excessive focus on semantics may overwhelm complementary information. Thus, learned representations become less expressive and less capable of capturing diverse but contextually relevant library associations.

Semantic constraints in RAHGRec act as regulators of higher-order propagation rather than simple amplification mechanisms. The balanced setting $\alpha_p = \alpha_l = \lambda_p = \lambda_l = 0.50$ gives the strongest performance by filtering noisy relations while retaining sufficient structural and semantic diversity for accurate library recommendation.

5.5 RQ4: How does latent embedding dimensionality affect RAHGRec’s ability to capture multi-scale structural and semantic signals without overfitting?

The dimensions of the latent space play an important role in determining the expressiveness of a representation learning model. In the proposed approach, the number of dimensions of the embedding d is responsible for the model’s ability to capture various kinds of signals that arise from high order hypergraph structures, semantics, and behavior-based interactions. Too small latent space will not provide the required expressive power to represent such dependencies while too large one can result in unnecessary feature accumulation and over-parametrization in the case of insufficient training signal.

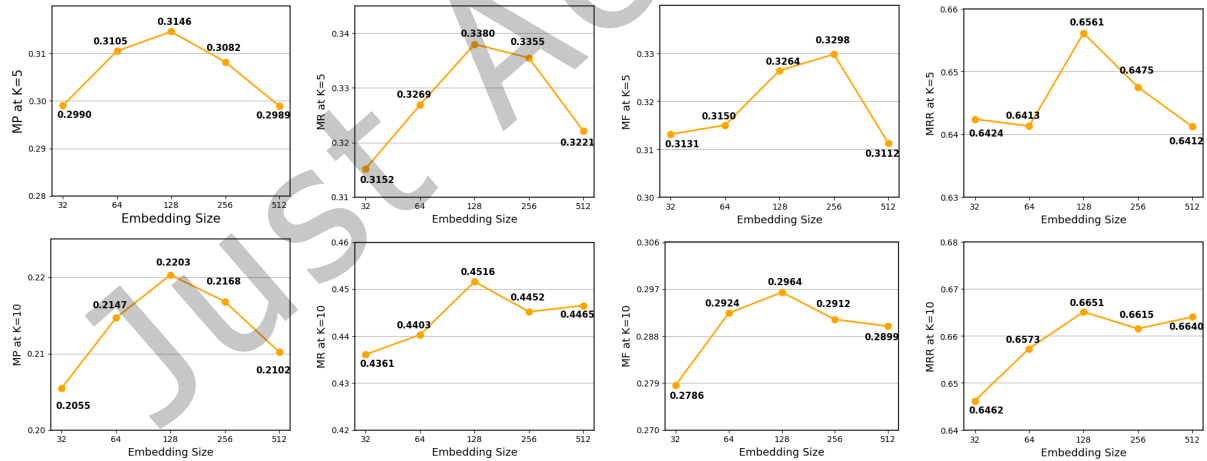


Fig. 6. Impact of latent space dimensionality (d) on RAHGRec performance metrics for the $r_m = 60\%$ dataset at $K = 5$ and $K = 10$.

In order to evaluate this trade-off, we will change the dimensionality d of embeddings from 32 to 512 and measure the recommendation performance under a challenging sparsity level ($r_m = 60\%$). This regime places

greater pressure on the representation, with fewer observed interactions, the model must rely more on semantic and structural generalization rather than direct evidence. We report MP, MR, MF, and MRR at $K = 5$ and $K = 10$, and summarize the results in Figure 6.

The findings demonstrate a uniform trend for all the metrics. When using a small value of the embedding size ($d = 32$), the proposed approach achieves relatively poor performance, which is consistent with an under-parameterized representation. For example, at $K = 5$, MP and MR are equal to 0.2990 and 0.3152, respectively, demonstrating that the latent space lacks the capacity to incorporate various structural and semantic cues contained in the dataset. As d increases, performance improves steadily, indicating that the model is better able to capture multi-hop co-usage structure and role-specific semantic cues.

We observe the best overall performance at $d = 128$, where RAHGRec achieves the highest scores across all evaluation metrics. At this dimension, the model achieves MP = 0.3146, MR = 0.3380, MF = 0.3264, and MRR = 0.5901 for $K = 5$, with similarly strong gains observed at $K = 10$. These findings suggest that a moderately sized embedding space is sufficient to represent the complexity of both higher-order hypergraph structures and knowledge graph semantics, while still supporting robust generalization under sparse conditions. This peak indicates that an intermediate-dimensional latent space can capture local usage patterns, higher-order hypergraph co-adoption, and role-specific semantic signals across multiple propagation scales, without compressing these signals into overly limited representations.

From this point, any further increase in embedding dimension does not yield much improvement, and eventually, there comes a stage where it starts to have an adverse effect on the results. For example, at $d = 512$, MP and MR become 0.2989 and 0.3221 at $K = 5$, respectively, while MRR also takes a slight dip. It seems as if larger embeddings add some sort of redundancy or noise in addition to useful information. In such settings, over-parameterization can make optimization more difficult and increase the risk of overfitting.

A moderate embedding dimensionality offers the best balance between expressiveness and stability in RAHGRec. In particular, the setting $d = 128$ achieves the strongest overall performance because it can capture multi-scale structural and semantic signals effectively. In contrast, larger embeddings tend to introduce redundant information and increase the risk of overfitting when supervision is sparse.

5.6 RQ5: How robust and scalable is RAHGRec under larger datasets, sparse interactions, and long-tail project distributions, while maintaining computational efficiency?

Beyond achieving high predictive accuracy, a practical recommendation system must also work reliably in large and continuously evolving software ecosystems, while keeping performance stable and computational costs reasonable. This research question therefore examines the robustness and scalability of the proposed approach from two perspectives: recommendation quality and system efficiency. Specifically, it asks whether the model's performance gains remain consistent as the amount of data increases, and whether these gains can be achieved without placing excessive computational demands on the system.

To this end, we experimentally evaluate the proposed approach on increasingly large datasets, ranging from 100 to 4,000 projects, while keeping the interaction sparsity fixed at $r_m = 60\%$. This highly sparse setting reflects a realistic long-tail scenario, where many projects have limited interaction histories and recommendation signals are unevenly distributed. In such cases, effective recommendations cannot depend only on direct co-adoption signals, but must also draw on higher-order structural patterns and semantic generalization. The quality of the recommendations is evaluated using MP, MR, MF and MRR at $K = 5$, $K = 10$, and $K = 20$, together with the corresponding training time, as reported in Table 5.

Table 5. Performance Comparison of RAHGRec for Different Dataset Sizes ($r_m = 60\%$)

Dataset Size	K = 5				K = 10				K = 20				Training Time (s)
	MP	MR	MF	MRR	MP	MR	MF	MRR	MP	MR	MF	MRR	
100	0.1420	0.1650	0.1512	0.4012	0.1120	0.2814	0.1633	0.4235	0.0715	0.3251	0.1189	0.4352	950
500	0.1981	0.2226	0.2070	0.4825	0.1456	0.3390	0.1985	0.5058	0.0972	0.4287	0.1582	0.5216	2800
1,000	0.2375	0.2614	0.2489	0.5372	0.1704	0.3806	0.2304	0.5683	0.1119	0.4750	0.1750	0.5897	5100
2,000	0.2684	0.2939	0.2805	0.5913	0.1912	0.4159	0.2601	0.6291	0.1248	0.5150	0.1944	0.6460	9600
4,000	0.3052	0.3308	0.3175	0.6349	0.2149	0.4448	0.2832	0.6544	0.1372	0.5432	0.2122	0.6621	13200

Unlike the full recommendation-quality comparison in RQ1, Table 6 focuses on computational cost. For this reason, we include the five most recent and closely related baselines from Table 3. These methods are the most relevant for this comparison because they involve learned representations, graph or hypergraph propagation, or semantic modeling, which makes their training time and memory usage more directly comparable with RAHGRec. The two remaining baselines are still included in the full effectiveness comparison in Table 3. Therefore, their exclusion from Table 6 does not affect the completeness of the recommendation-quality evaluation.

Table 6. Computational Complexity Comparison

Model	Training Time (s)	Memory Usage (MB)
RAHGRec	15,056	1952
MsRec [13]	8,737.55	2733.23
PyRec [21]	3,061.45	3,428
HCF [3]	2964.6	1130
GRec [20]	1,347.3	418
LibSeek [11]	314.1	12,670

The results demonstrate that RAHGRec exhibits stable and monotonic performance improvements as the size of the data set increases. For example, when $K = 5$, the mean precision improves from 0.1420 using 100 projects to 0.3052 when using 4,000 projects, while the mean recall improves from 0.1650 to 0.3308 in the same interval. The results obtained are the same for both MF and MRR, where the MRR goes up from 0.4012 to 0.6349. The consistency of the improvements indicates that RAHGRec gradually utilizes more abundant structural and semantic information with more data added without suffering from performance bottlenecks.

Notably, these performance gains remain stable even as the recommendation list length changes. At $K = 10$ and $K = 20$, the model continues to achieve consistent improvements across both accuracy-oriented and ranking-based metrics, suggesting that the learned representations generalize well to larger candidate sets. This pattern highlights the robustness of the role-aware hypergraph construction and gated semantic propagation modules, which continue to scale effectively as more projects and libraries, especially long-tail ones, are added to the ecosystem.

From a computational complexity perspective, the training time reported in Table 5 increases with dataset size, showing a steady and predictable scaling pattern. Although RAHGRec requires more computation than several lighter baselines, this additional cost mainly comes from its explicit modeling of higher-order hyperedges and role-aware semantic constraints, rather than from optimization inefficiency. To make this trade-off clearer, Table 6 compares the training time and memory usage of representative methods.

Compared with PyRec and MsRec, RAHGRec takes longer to train, but it uses less memory than both methods. This suggests that the proposed sparse role-aware hypergraph structure introduces additional computation while keeping memory consumption controlled. Compared with HCF and GRec, RAHGRec requires more time and

memory because it jointly models higher-order hyperedges and typed semantic relations. LibSeek trains quickly, but it consumes substantially more memory than all other methods. Overall, Table 6 shows that RAHGRec incurs additional training cost in exchange for richer structural and semantic modeling, while maintaining a manageable memory footprint.

RAHGRec remains robust as the dataset size increases, even under sparse interactions and long-tail project distributions. Its recommendation quality improves steadily as more data become available, while the additional training cost is mainly tied to its richer higher-order and role-aware semantic modeling. At the same time, its memory usage remains manageable.

5.7 RQ6: How do the major components of RAHGRec contribute to recommendation performance?

To answer this research question, we conducted a systematic ablation study in which one key module was removed or simplified at a time, while all other experimental settings were kept unchanged. This analysis helps isolate the contribution of each design choice and shows how different structural and semantic inductive biases influence recommendation accuracy and ranking performance, especially under sparse data conditions.

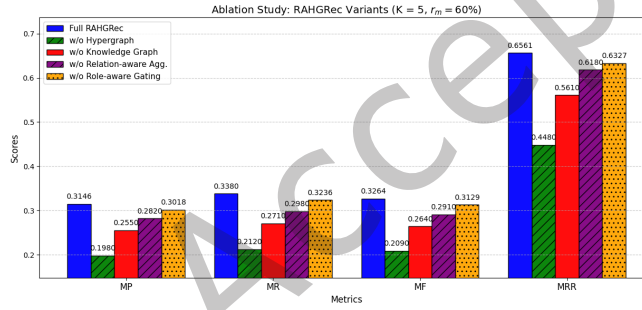


Fig. 7. Ablation Study: Performance Impact of Removing Key Components from RAHGRec at $r_m = 60\%$, $K = 5$.

The complete proposed system combines four interrelated components. First, *hypergraph-based modeling*, which reveals the higher-order patterns of co-adoptions of projects and libraries. Second, *knowledge graph integration*, which injects semantic information such as dependencies, topics, and maintainers. Third, *role-aware gating*, which regulates the flow of information according to semantic roles. Fourth, *relation-aware aggregation*, which dynamically adjusts the message weight during representation learning. Each component of the system of the full proposed approach enables it to reason about both the high-level structural patterns and the context semantics, while simultaneously ignoring information that is irrelevant or contradictory to the context.

Based on this design, we define the following ablated variants:

- **w/o Hypergraph:** removes the hypergraph structure and reduces the model to a standard pairwise project-library graph, thereby eliminating any modeling of higher-order co-adoption.
- **w/o Knowledge Graph:** excludes all semantic relations, forcing the model to rely solely on structural co-usage information.
- **w/o Relation-aware Aggregation:** replaces adaptive aggregation with straightforward mean pooling, thus discarding relation-specific weighting.

- **w/o Role-aware Gating:** collapses all semantic roles into a single unified channel, disabling role-specific control while retaining both the hypergraph structure and semantic features.

All ablation experiments are conducted under a challenging sparsity setting of $r_m = 60\%$ and are assessed at $K = 5$, capturing real-world situations where interaction records are partial and delivering accurate top- K recommendations really matters. We evaluate performance using mean precision (MP), mean recall (MR), mean F1 score (MF), and mean reciprocal rank (MRR).

As shown in Figure 7, the complete proposed model achieves the best performance across all metrics, with MP = 0.3146, MR = 0.3380, MF = 0.3264, and MRR = 0.6561. The ablation results further show that the hypergraph component plays the most critical role in the model’s effectiveness. In particular, removing this component leads to a substantial drop in performance, reducing MR to 0.2120 and MRR to 0.4480.

Removing the knowledge graph results in a significant performance drop (MR = 0.2710, MRR = 0.5610), indicating that semantic information is crucial in identifying libraries with similar usage rates but different roles or ways of depending on each other. Replacement of relation-aware aggregation with simple and uniform pooling results in a moderate but constant decrease in ranking performance (MRR = 0.6180), indicating that adaptive weighting is helpful in highlighting significant relations and minimizing noisy interactions.

Removing role-aware gating also degrades performance across the evaluation metrics, with MRR decreasing to 0.6327, even though the underlying hypergraph structure and semantic features are still retained. This result shows that simply injecting semantic information is not sufficient. Role-aware control is needed to prevent conflicting semantic cues from propagating through higher-order structures. The gap between the full model and this ablated variant supports the importance of active semantic role regulation in controlling higher-order information propagation.

The ablation study shows that each major component contributes to the effectiveness of RAHGRec. Hypergraph modeling provides the largest structural benefit, knowledge graph integration improves contextual understanding, relation-aware aggregation strengthens ranking quality, and role-aware gating preserves semantic consistency during higher-order propagation.

5.8 Threats to Validity

Although our experimental results show that RAHGRec is both effective and robust, we acknowledge that there are several potential threats to validity that may influence how our findings are interpreted, generalized, or replicated. We examine these threats to validity in line with well-established guidelines from empirical software engineering.

Internal Validity Internal validity addresses whether the observed performance gains can legitimately be attributed to the proposed design decisions rather than to confounding influences. Our experiments make use of a publicly available, Python-focused dataset, which may contain inherent biases related to how repositories are selected, which libraries are most frequently used, or the differing levels of project activity. To mitigate the effects of these factors, we conduct multiple random data splits and evaluate performance under several sparsity conditions and recommendation cutoffs. In addition, all baseline methods are either reimplemented or configured in accordance with their original descriptions and are tested using the same data partitions and evaluation metrics. Nevertheless, some hyperparameters of RAHGRec, such as embedding dimensionality, propagation depth, and similarity thresholds, are determined empirically and may not represent globally optimal choices for every dataset.

External Validity The External validity is concerned with how well our results generalize to other situations than the precise one we investigated. We chose to focus our research on Python projects and project libraries,

and as such, the performance we see here is not necessarily going to scale directly to other ecosystems such as Java, Android, or graphs involving multiple languages. The management of dependencies, the size of the ecosystem, and the presence of semantic metadata can all impact the performance of the model. Our scalability results indicate that the trends continue as the size of the dataset increases, but in order to make a truly general claim about the proposed approach, we should investigate it in a broader range of software ecosystems.

Construct Validity Construct validity concerns whether our evaluation measures truly reflect what we intend to study. We report standard recommendation metrics (MP, MR, MF, MRR, and coverage), which are commonly used in recommender-systems research and in software reuse studies. These metrics are useful for evaluating the ranking quality and effectiveness of each technique in identifying candidate libraries using implicit feedback. Yet, these measures fail to consider other aspects that could influence the decisions made by a developer, including integration cost, learning time, and maintenance cost. We also use Friedman and Wilcoxon tests, reported in Table 4, to examine whether the performance differences among methods are statistically reliable under our evaluation protocol. Nevertheless, these tests only reflect differences on the selected benchmark metrics, and they do not directly capture the practical trade-offs that often matter in real software projects beyond ranking accuracy and coverage.

Reproducibility Reproducibility is especially important when working with complex machine learning models. To support independent verification and make reuse easier, we release the full source code for RAHGREG, along with all scripts used for data preprocessing. The experimental setup and datasets are publicly available, and we document the computing environment in detail to reduce ambiguity for researchers who wish to replicate our results. Even so, small numerical differences may still arise because of variations in hardware, software configurations, or random seeds.

By identifying and assessing these risks, we provide a more rigorous evaluation of the proposed method, strengthen confidence in our results, and clarify directions for future work.

6 Discussion

The experiments show that our proposed approach outperforms state-of-the-art methods for project-library recommendation across different sparsity levels, cutoff list sizes, and evaluation metrics. These improvements are not simply the result of greater model capacity. They also suggest that the model can learn higher-order co-adoption patterns and role-based semantic meanings among libraries. In this way, the model captures useful signals that are difficult to learn from pairwise graphs or homogeneous embedding representations alone.

A central finding of this study is that higher-order structural information and semantic knowledge are complementary rather than interchangeable. The role-aware hypergraph module captures library adoption behavior at the group level, which is especially useful when direct co-use evidence is limited. At the same time, the knowledge graph provides semantic information through dependency relations, topic similarities, and authorship signals, guiding representation learning toward libraries that are more relevant to the project context. The strong performance of the proposed approach under limited usage information suggests that these two perspectives offer distinct but mutually reinforcing evidence.

The sparsity analysis further clarifies the implications of these findings. In our experiments, the sparsity parameter controls the proportion of observed project-library interactions removed from the training data, thereby modeling incomplete adoption histories. Such incomplete histories are common in real software ecosystems because dependency information may be affected by refactoring, modularization, partial observation windows, or incomplete metadata. The fact that RAHGREG remains effective under these conditions suggests that role-aware semantic propagation can help compensate for limited interaction evidence. This interpretation keeps the scope of our conclusions aligned with the masked sparse recommendation setting evaluated in the experiments.

Sensitivity analyses help explain the trade-offs behind the main design choices in the proposed framework. The results for RQ2 and RQ4 show that increasing the propagation depth or embedding dimensionality can improve performance, but only up to a certain point. After a modest threshold, the gains begin to level off. This suggests that the proposed method can capture useful higher-order relationships without requiring very deep propagation, which may otherwise lead to over-smoothing or redundant representations. Similarly, the results for RQ3 show the importance of carefully setting similarity thresholds and assigning appropriate weights to semantic information, so that noisy relations are filtered out while meaningful ones are retained.

In terms of scalability, the RQ5 analysis shows that RAHGRec continues to outperform the baselines as the dataset becomes larger. Its training time and memory usage also grow in a steady and manageable manner. Although the framework introduces additional computational cost compared with simpler graph-based methods, this overhead mainly comes from its richer modeling of higher-order interactions and semantic roles. The use of sparse hypergraph operators helps keep the method practical even for large-scale software ecosystems.

Another important finding is that RAHGRec consistently improves coverage. This means that the system does not only favor popular libraries, but is also better able to identify less frequently recommended libraries that are still relevant to a given project. This is important in real-world development, where developers often need libraries that match specific functional and contextual requirements rather than libraries that are merely popular. The ablation study supports this observation, showing that removing either the hypergraph framework or the semantic components reduces coverage.

From a practitioner perspective, these results suggest that improved ranking quality can reduce the search space that developers need to inspect when selecting libraries for a project. By prioritizing libraries that are both frequently co-adopted and semantically compatible with the project context, RAHGRec can help developers identify suitable candidates earlier in the selection process. This can support practical tasks such as comparing alternative libraries, checking dependency compatibility, and reasoning about functional fit or maintenance context. In AI-assisted development workflows, RAHGRec can be viewed as a context-aware retrieval layer for dependency suggestion: given a project context, it can rank candidate libraries using higher-order co-adoption evidence and typed semantic relations before these candidates are surfaced to a developer or passed to a coding assistant or retrieval-augmented generation pipeline. This could help AI-assisted tools move beyond popularity-based or surface-level similarity signals when suggesting dependencies. However, these implications should be interpreted as potential practical benefits rather than direct deployment evidence, since the current evaluation is based on offline ranking metrics and does not directly measure developer effort, API learning cost, integration difficulty, coding-agent behavior, or long-term maintenance outcomes.

In summary, the framework finds an effective balance between the expressiveness of the representation and its computational efficiency, thus providing an interesting way forward for library recommendation in practical software ecosystems. Although our study focuses on static usage patterns in one software ecosystem, further research can include time-related adoption patterns, developer feedback and knowledge graphs across several ecosystems. This will allow the framework to be more aware of changes in dependency practices and will help understand the evolution of dependencies within software.

7 Conclusion and Future Work

We presented RAHGRec, a role-aware hypergraph learning approach for library recommendation, which effectively combines high order co-adoption information and structured semantic information. Retaining semantic roles in the representation learning, our framework moves beyond the pairwise usage patterns and helps to diffuse controlled semantic context information in the software ecosystem, helping RAHGRec leverage global popularity and semantic consistency information while recommending libraries.

Although the results are encouraging, several directions remain for extending and strengthening this line of work. First, the current framework is based on static snapshots of ecosystem data. Incorporating temporal dynamics would allow the model to capture evolving dependency patterns, library lifecycles, and emerging trends more effectively. Second, although this study focuses on the Python ecosystem, applying the approach to other programming languages and package managers would help evaluate its generality and reveal ecosystem-specific challenges. Third, integrating developer feedback or downstream tooling signals could support more heterogeneous, interactive, and context-sensitive recommendations. Finally, future work could explore simplified or approximate architectures to further reduce computational cost and move the framework closer to real-time deployment and larger-scale applications. Overall, RAHGREC shows that combining role-aware semantic reasoning with higher-order structural modeling is a promising direction for improving library recommendation. We hope this work provides a foundation for future research on semantically grounded and scalable recommendation systems in software engineering ecosystems.

Acknowledgment

The authors acknowledge the funding received under the SPARC scheme of the Government of India.

References

- [1] Yoav Benjamini and Yosef Hochberg. 1995. Controlling the false discovery rate: a practical and powerful approach to multiple testing. *Journal of the Royal statistical society: series B (Methodological)* 57, 1 (1995), 289–300.
- [2] Fidel Cacheda, Víctor Carneiro, Diego Fernández, and Vreixo Formoso. 2011. Comparison of collaborative filtering algorithms: Limitations of current techniques and proposals for scalable, high-performance recommender systems. *ACM Transactions on the Web (TWEB)* 5, 1 (2011), 1–33.
- [3] Lianrong Chen, Naidan Mei, Yingying He, Wanping Liu, Guo Zhong, and Mingdong Tang. 2023. High-order collaborative filtering for third-party library recommendation. In *2023 IEEE International Conference on Web Services (ICWS)*. IEEE, 483–492.
- [4] Moataz Chouchen, Ali Ouni, and Mohamed Wiem Mkaouer. 2020. Androlib: Third-party software library recommendation for android applications. In *International Conference on Software and Software Reuse*. Springer, 208–225.
- [5] Chenjiao Feng, Jiye Liang, Peng Song, and Zhiqiang Wang. 2020. A fusion collaborative filtering method for sparse data in recommender systems. *Information Sciences* 521 (2020), 365–379.
- [6] Yifan Feng, Haoxuan You, Zizhao Zhang, Rongrong Ji, and Yue Gao. 2019. Hypergraph neural networks. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 33. 3558–3565.
- [7] Milton Friedman. 1937. The use of ranks to avoid the assumption of normality implicit in the analysis of variance. *Journal of the american statistical association* 32, 200 (1937), 675–701.
- [8] Yue Gao, Yifan Feng, Shuyi Ji, and Rongrong Ji. 2022. HGNN+: General hypergraph neural networks. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 45, 3 (2022), 3181–3199.
- [9] Xavier Glorot and Yoshua Bengio. 2010. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the thirteenth international conference on artificial intelligence and statistics*. JMLR Workshop and Conference Proceedings, 249–256.
- [10] Qingyu Guo, Fuzhen Zhuang, Chuan Qin, Hengshu Zhu, Xing Xie, Hui Xiong, and Qing He. 2020. A survey on knowledge graph-based recommender systems. *IEEE Transactions on Knowledge and Data Engineering* 34, 8 (2020), 3549–3568.
- [11] Qiang He, Bo Li, Feifei Chen, John Grundy, Xin Xia, and Yun Yang. 2020. Diversified third-party library prediction for mobile app development. *IEEE Transactions on Software Engineering* 48, 1 (2020), 150–165.
- [12] Yifan Hu, Yehuda Koren, and Chris Volinsky. 2008. Collaborative filtering for implicit feedback datasets. In *2008 Eighth IEEE international conference on data mining*. Ieee, 263–272.
- [13] Abhinav Jamwal and Sandeep Kumar. 2026. A multi-scale hypergraph-based approach for third-party library recommendation in mobile app development. *ACM Transactions on Software Engineering and Methodology* 35, 6, 1–28.
- [14] Maurice G Kendall and B Babington Smith. 1939. The problem of m rankings. *The annals of mathematical statistics* 10, 3 (1939), 275–287.
- [15] Diederik P Kingma and Jimmy Ba. 2014. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980* (2014).
- [16] Thomas N Kipf and Max Welling. 2016. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907* (2016).
- [17] Dezhen Kong, Qiuyuan Chen, Lingfeng Bao, Chenxing Sun, Xin Xia, and Shanping Li. 2022. Recommending code reviewers for proprietary software projects: A large scale study. In *2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 630–640.

- [18] Yehuda Koren, Robert Bell, and Chris Volinsky. 2009. Matrix factorization techniques for recommender systems. *Computer* 42, 8 (2009), 30–37.
- [19] Daniel Lee and H Sebastian Seung. 2000. Algorithms for non-negative matrix factorization. *Advances in neural information processing systems* 13 (2000).
- [20] Bo Li, Qiang He, Feifei Chen, Xin Xia, Li Li, John Grundy, and Yun Yang. 2021. Embedding app-library graph for neural third party library recommendation. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 466–477.
- [21] Bo Li, Haowei Quan, Jiawei Wang, Pei Liu, Haipeng Cai, Yuan Miao, Yun Yang, and Li Li. 2024. Neural Library Recommendation by Embedding Project-Library Knowledge Graph. *IEEE Transactions on Software Engineering* (2024).
- [22] Mengran Li, Yong Zhang, Xiaoyong Li, Yuchen Zhang, and Baocai Yin. 2023. Hypergraph transformer neural networks. *ACM Transactions on Knowledge Discovery from Data* 17, 5 (2023), 1–22.
- [23] Xiaoyu Liu, LiGuo Huang, and Vincent Ng. 2018. Effective API recommendation without historical software repositories. In *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*. 282–292.
- [24] Zhongxin Liu, Xin Xia, David Lo, and John Grundy. 2019. Automatic, highly accurate app permission recommendation. *Automated Software Engineering* 26 (2019), 241–274.
- [25] Christopher D Manning, Prabhakar Raghavan, and Hinrich Schütze. 2008. *Introduction to information retrieval*. Cambridge university press.
- [26] Senthilselvan Natarajan, Subramaniaswamy Vairavasundaram, Sivaramakrishnan Natarajan, and Amir H Gandomi. 2020. Resolving data sparsity and cold start problem in collaborative filtering recommender system using linked open data. *Expert Systems with Applications* 149 (2020), 113248.
- [27] Phuong T Nguyen, Juri Di Rocco, Davide Di Ruscio, and Massimiliano Di Penta. 2020. CrossRec: Supporting software developers by recommending third-party libraries. *Journal of Systems and Software* 161 (2020), 110460.
- [28] Phuong T Nguyen, Juri Di Rocco, Davide Di Ruscio, Lina Ochoa, Thomas Degueule, and Massimiliano Di Penta. 2019. Focus: A recommender system for mining api function calls and usage patterns. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, 1050–1060.
- [29] Xiaoxue Ren, Xinyuan Ye, Zhenchang Xing, Xin Xia, Xiwei Xu, Liming Zhu, and Jianling Sun. 2020. API-misuse detection driven by fine-grained API-constraint knowledge graph. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*. 461–472.
- [30] Steffen Rendle, Christoph Freudenthaler, Zeno Gantner, and Lars Schmidt-Thieme. 2012. BPR: Bayesian personalized ranking from implicit feedback. *arXiv preprint arXiv:1205.2618* (2012).
- [31] Wei Tang, Zhengzi Xu, Chengwei Liu, Jiahui Wu, Shouguo Yang, Yi Li, Ping Luo, and Yang Liu. 2022. Towards understanding third-party library dependency in c/c++ ecosystem. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*. 1–12.
- [32] Ferdian Thung, David Lo, and Julia Lawall. 2013. Automated library recommendation. In *2013 20th Working conference on reverse engineering (WCRE)*. IEEE, 182–191.
- [33] Maolin Wang, Yaoming Zhen, Yu Pan, Yao Zhao, Chenyi Zhuang, Zenglin Xu, Ruocheng Guo, and Xiangyu Zhao. 2024. Tensorized hypergraph neural networks. In *Proceedings of the 2024 SIAM International Conference on Data Mining (SDM)*. SIAM, 127–135.
- [34] Xiang Wang, Xiangnan He, Yixin Cao, Meng Liu, and Tat-Seng Chua. 2019. Kgat: Knowledge graph attention network for recommendation. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*. 950–958.
- [35] Xiang Wang, Xiangnan He, Meng Wang, Fuli Feng, and Tat-Seng Chua. 2019. Neural graph collaborative filtering. In *Proceedings of the 42nd international ACM SIGIR conference on Research and development in Information Retrieval*. 165–174.
- [36] Frank Wilcoxon. 1945. Individual comparisons by ranking methods. *Biometrics bulletin* 1, 6 (1945), 80–83.
- [37] Xinqiang Xie, Xiaochun Yang, Bin Wang, and Qiang He. 2021. DevRec: Multi-relationship embedded software developer recommendation. *IEEE Transactions on Software Engineering* 48, 11 (2021), 4357–4379.
- [38] Huan Yu, Xin Xia, Xiaoqiong Zhao, and Weiwei Qiu. 2017. Combining collaborative filtering and topic modeling for more accurate android mobile app library recommendation. In *Proceedings of the 9th Asia-Pacific Symposium on Internetwork*. 1–6.
- [39] Yanjie Zhao, Li Li, Xiaoyu Sun, Pei Liu, and John Grundy. 2021. Icon2Code: Recommending code implementations for Android GUI components. *Information and Software Technology* 138 (2021), 106619.
- [40] Yanjie Zhao, Li Li, Haoyu Wang, Qiang He, and John Grundy. 2022. APIMatchmaker: Matching the right APIs for supporting the development of Android apps. *IEEE Transactions on Software Engineering* 49, 1 (2022), 113–130.
- [41] Yu Zhao, Tingting Yu, Ting Su, Yang Liu, Wei Zheng, Jingzhi Zhang, and William G.J. Halfond. 2019. ReCDroid: Automatically Reproducing Android Application Crashes from Bug Reports. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. 128–139. doi:10.1109/ICSE.2019.00030
- [42] Dengyong Zhou, Jiayuan Huang, and Bernhard Schölkopf. 2006. Learning with hypergraphs: Clustering, classification, and embedding. *Advances in neural information processing systems* 19 (2006).